

| Instruction Processor |                                                             |
|-----------------------|-------------------------------------------------------------|
| NOP                   | No Operation                                                |
| CLRL                  | Clear extended Load Register to zeroes.                     |
| MACX,L                | Execute Load Register as a Macro                            |
| PVMAC                 | Switch macro specified in Macro address register to Private |
| GPMAC                 | Switch macro specified in Macro address register to Group   |
| WREG,DP               | Write to External Hex Display                               |
| RREG,L                | Read from Load Register                                     |
| WREG,L                | Write to Load Register                                      |
| END                   | Immediately end this instruction cycle.                     |
|                       |                                                             |
|                       |                                                             |
|                       |                                                             |
| LDW***                | Shift 16-bit word ##### into Load Register                  |
| LDB**                 | Shift byte ## into Load Register                            |
| WRMAC**               | Write Load Register into Macro slot ##                      |
| MACX,0-15             | Execute selected Macro 0 to 15                              |

| Register Bank |                                          |
|---------------|------------------------------------------|
| RREG,0-9      | Read from Group or Private Register 0-9  |
| WREG,0-9      | Write to Group or Private Register 0-9   |
| RREG,A        | Read from Group Register A               |
| WREG,A        | Write to Group Register A                |
| PVREG         | Switch last read Register 0-9 to Private |
| GPREG         | Switch last read Register 0-9 to Group   |

| Program Counter Control & Memory |                                                                                           |
|----------------------------------|-------------------------------------------------------------------------------------------|
| JMP***                           | Perform PC Jump to Memory Address Register location                                       |
| JSR***                           | Perform PC Subroutine Jump to Memory Address Register location                            |
| RET                              | Return from Subroutine to stored JSR location (Small stack, 4 levels of recursion avail.) |
| JRST                             | Clear JSR Stack for this process                                                          |
| LOOP                             | Record a loop start location (Small stack, 4 levels of recursion avail.)                  |
| LRET                             | Return to loop start location for next process instruction cycle.                         |
| LBRK                             | End loop and pull loop address off of loop stack. Execution continues at next inst. word. |
| LRST                             | Clear Loop Stack for this process                                                         |
| REDO                             | For this process, the next instruction cycle will repeat at this current location.        |
| MREAD                            | Read from Memory location specified by Memory Address Register                            |
| MWRITE                           | Write to Memory location specified by Memory Address Register                             |
| RSTMA                            | Reset Memory Address Register to initial address location. (PC or Interface addr.)        |
| HOP,RF                           | Move Memory Address Register Relative Forward                                             |
| HOP,RR                           | Move Memory Address Register Relative Right                                               |
| HOP,RL                           | Move Memory Address Register Relative Left                                                |
| HOP,RB                           | Move Memory Address Register Relative Reverse                                             |
| HOP,AF                           | Move Memory Address Register to Absolute Forward Direction                                |
| HOP,AR                           | Move Memory Address Register to Absolute Right Direction                                  |
| HOP,AL                           | Move Memory Address Register to Absolute Left Direction                                   |
| HOP,AB                           | Move Memory Address Register to Absolute Rear Direction                                   |
| RREG,MA                          | Read from full Memory Address Register                                                    |
| WREG,MA                          | Write to full Memory Address Register                                                     |
| RREG,MX                          | Read from Memory Address Register, X field                                                |
| WREG,MX                          | Write to Memory Address Register, X field                                                 |
| RREG,MY                          | Read from Memory Address Register, Y field                                                |
| WREG,MY                          | Write to Memory Address Register, Y field                                                 |
| RREG,MP                          | Read from Memory Address Register, Page/Bank field                                        |
| WREG,MP                          | Write to Memory Address Register, Page/Bank field                                         |
| PAGE**                           | Select Address Page                                                                       |
| PDEF**                           | Define Address Page                                                                       |
|                                  |                                                                                           |
|                                  |                                                                                           |

| Arithmetic Logic Unit |                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------|
| WREG,UI               | Write to Unary ALU Input Register                                                      |
| WREG,B1               | Write to Binary ALU Input 1 Register                                                   |
| WREG,B2               | Write to Binary ALU Input 2 Register                                                   |
| WREG,AC               | Write to ALU Configuration Register                                                    |
| RREG,AC               | Read from ALU Configuration Register                                                   |
| RREG,UO               | Read from Unary ALU Output Register                                                    |
| RREG,BO               | Read from Binary ALU Output Register                                                   |
| RREG,Z                | Read zero from static Zero Register                                                    |
| ACOFF                 | Temporarily disable customized ALU Configuration and use default config.               |
| ACON                  | Re-enable customized ALU Configuration                                                 |
| ACLR                  | Clear all ALU Input/Output Registers (but not ALU config register)                     |
|                       |                                                                                        |
|                       |                                                                                        |
|                       |                                                                                        |
| **                    |                                                                                        |
| LSET**                | Set ALU I/O Looping Mode                                                               |
| PASS                  | Pass Unary Input to Output without change                                              |
| NOT                   | Perform Logical Not Operation on Unary ALU                                             |
| INC                   | Perform Arithmetic Increment Operation on Unary ALU                                    |
| DEC                   | Perform Arithmetic Decrement Operation on Unary ALU                                    |
| NEG                   | Perform Arithmetic Negate Operation on Unary ALU                                       |
| WSW                   | Perform Upper/Lower Word Swap on Unary ALU                                             |
| BRR                   | Perform Byte Rotate Right Operation on Unary ALU                                       |
| BRL                   | Perform Byte Rotate Left Operation on Unary ALU                                        |
| BSET                  | Perform Bit Set operation on Binary ALU Input 1 with bit position in Input 2           |
| BCLR                  | Perform Bit Clear operation on Binary ALU Input 1 with bit position in Input 2         |
| BINV                  | Perform Bit Invert operation on Binary ALU Input 1 with bit position in Input 2        |
| BTST                  | Perform Bit Test operation on Binary ALU Input 1 with bit position in Input 2          |
| ADD                   | Perform Arithmetic ADD operation on Binary ALU                                         |
| ADD,W****             | Immediate ADD 2-byte unsig. word to Binary ALU Input 1 and write back to ALU Input 1   |
| ADD,B1**              | Immediate ADD unsigned byte to Binary ALU Input 1 and write back to ALU Input 1        |
| ADD,B2**              | Immediate ADD unsigned byte to Binary ALU Input 2 and write back to ALU Input 2        |
| ROR                   | Perform Bit Rotate Right Operation on Unary ALU                                        |
| ROL                   | Perform Bit Rotate Left Operation on Unary ALU                                         |
| RRX                   | Perform Bit Rotate Right w/Carry Operation on Unary ALU                                |
| RLX                   | Perform Bit Rotate Left w/Carry Operation on Unary ALU                                 |
| LSR                   | Perform Bit Logical Shift Right Operation on Unary ALU                                 |
| LSL                   | Perform Bit Logical Shift Left Operation on Unary ALU                                  |
| ASR                   | Perform Arithmetic Shift Right Operation on Unary ALU                                  |
| BSL                   | Perform Byte Shift Left Operation on Unary ALU                                         |
| OR                    | Perform Logical OR operation on Binary ALU                                             |
| XOR                   | Perform Logical XOR operation on Binary ALU                                            |
| IMP                   | Perform Logical Imply operation on Binary ALU                                          |
| CMP                   | Perform Arithmetic Compare operation on Binary ALU                                     |
| SUB                   | Perform Arithmetic Subtract operation on Binary ALU                                    |
| SUB,W***              | Immediate Subtract 2-byte u.word from Binary ALU Input 1 and write back to ALU Input 1 |
| SUB,B1**              | Immediate unsigned Subtract byte from Binary ALU Input 1 and write back to ALU Input 1 |
| SUB,B2**              | Immediate unsigned Subtract byte from Binary ALU Input 2 and write back to ALU Input 2 |
| IB0                   | Isolate Byte 0 from Unary ALU Input and output to Byte 0 of Unary ALU Output           |
| IB1                   | Isolate Byte 1 from Unary ALU Input and output to Byte 0 of Unary ALU Output           |
| IB2                   | Isolate Byte 2 from Unary ALU Input and output to Byte 0 of Unary ALU Output           |
| IB3                   | Isolate Byte 3 from Unary ALU Input and output to Byte 0 of Unary ALU Output           |
| ZB0                   | Clear Byte 0 on Unary ALU Input and output to Unary ALU Output                         |
| ZB1                   | Clear Byte 1 on Unary ALU Input and output to Unary ALU Output                         |
| ZB2                   | Clear Byte 2 on Unary ALU Input and output to Unary ALU Output                         |
| ZB3                   | Clear Byte 3 on Unary ALU Input and output to Unary ALU Output                         |
| NOR                   | Perform Logical NOR operation on Binary ALU                                            |
| XNOR                  | Perform Logical XNOR operation on Binary ALU                                           |
| INH                   | Perform Logical Inhibit operation on Binary ALU                                        |
| NAND                  | Perform Logical NAND operation on Binary ALU                                           |
| AND                   | Perform Logical AND operation on Binary ALU                                            |
| AND,W****             | Immediate AND 2-byte unsig. word with Binary ALU Input 1 and write back to ALU Input 1 |
| AND,B1**              | Immediate AND unsigned byte with Binary ALU Input 1 and write back to ALU Input 1      |
| AND,B2**              | Immediate AND unsigned byte with Binary ALU Input 2 and write back to ALU Input 2      |

| Context Switching |                                                                                 |
|-------------------|---------------------------------------------------------------------------------|
| FORK,RF           | Fork Process with PCD Relative Forward Direction at current PC location         |
| FORK,RR           | Fork Process with PCD Relative Right Rotation at current PC location            |
| FORK,RL           | Fork Process with PCD Relative Left Rotation at current PC location             |
| FORK,RB           | Fork Process with PCD Relative Reversed Direction at current PC location        |
| FORK,AF           | Fork Process with PCD Absolute Forward Direction at current PC location         |
| FORK,AR           | Fork Process with PCD Absolute Right Direction at current PC location           |
| FORK,AL           | Fork Process with PCD Absolute Left Direction at current PC location            |
| FORK,AB           | Fork Process with PCD Absolute Rear Direction at current PC location            |
| FORK,MA           | Fork Process with PCD Relative Forward at Memory Address Location               |
| TERM              | Terminate Process Immediately.                                                  |
| DUPE              | Duplicate Process Queue Scheduling Entry and Increment Dupe Counter for Process |
| TRMD              | Terminate Duplicate Process Queue Scheduling Entry and Decrement Dupe Counter   |
| RREG,AP           | Read from Active/Dupe Process Counter Register                                  |
| RREG,EP           | Read from Extended Process Info Register                                        |
| HALT              | Stop process execution on CPU                                                   |
| RESET             | Stop process execution and reinitialize CPU                                     |

| Conditional & Flag Subsystem |                                                                               |
|------------------------------|-------------------------------------------------------------------------------|
| CLRA                         | Clear Flag 'A'                                                                |
| SETA                         | Set Flag 'A'                                                                  |
| CLRB                         | Clear Flag 'B'                                                                |
| SETB                         | Set Flag 'B'                                                                  |
| CLR X                        | Clear Flag 'X'                                                                |
| SETX                         | Set Flag 'X'                                                                  |
| CLRF**                       | Clear Specified Flag                                                          |
| SETF**                       | Set Specified Flag                                                            |
| RREG,FS0                     | Read Flag State Register 0 with Flags 0-31                                    |
| RREG,FS1                     | Read Flag State Register 1 with Flags 32-63                                   |
| PRVF                         | Make selected flag private (A,B,X, and G0-GF only)                            |
| GRPF                         | Make selected flag group access (A,B,X, and G0-GF only)                       |
| CLRC                         | Clear Carry Flag                                                              |
| SETC                         | Set Carry Flag                                                                |
| FSEL**                       | Select Specified Flag                                                         |
| COND**                       | Specify Flag for Conditional Operation (use second byte for flag numeric ID)  |
| COND,□                       | Specify Flag for conditional operation (specify flag character or numeric ID) |

| Conditional Byte 3/7 |                                                                   |
|----------------------|-------------------------------------------------------------------|
| ROT,□                | Unconditionally rotate PCD                                        |
| □,RR                 | Relative Right                                                    |
| □,RL                 | Relative Left                                                     |
| □,RB                 | Relative Back                                                     |
| □,AF                 | Absolute Forward                                                  |
| □,AR                 | Absolute Right                                                    |
| □,AL                 | Absolute Left                                                     |
| □,AB                 | Absolute Back                                                     |
|                      |                                                                   |
| CST,□                | Use selected flag, compare with true, run action if success       |
| CSF,□                | Use selected flag, compare with false, run action if success      |
| CAT,□                | Use auto-selected flag, compare with true, run action if success  |
| CAF,□                | Use auto-selected flag, compare with false, run action if success |
| □,NOP                | No operation                                                      |
| □,ROT,RR             | Rotate Relative Right                                             |
| □,ROT,RL             | Rotate Relative Left                                              |
| □,ROT,RB             | Rotate Relative Back (reverse)                                    |
| □,ROT,AF             | Rotate Absolute Forward                                           |
| □,ROT,AR             | Rotate Absolute Right                                             |
| □,ROT,AL             | Rotate Absolute Left                                              |
| □,ROT,AB             | Rotate Absolute Back                                              |
| □,REDO               | REDO opcode                                                       |
| □,TERM               | TERM opcode                                                       |
| □,LRET               | LRET opcode                                                       |
| □,LBRK               | LBRK opcode                                                       |
| □,RET                | RET opcode                                                        |
|                      |                                                                   |
|                      |                                                                   |

\*\* Two-byte opcode: Second byte is data.    \*\*\* Three-byte opcode: Second and third bytes are data.

|      | 0x0_    | 0x1_    | 0x2_   | 0x3_   | 0x4_    | 0x5_    | 0x6_   | 0x7_     | 0x8_    | 0x9_     | 0xA_     | 0xB_     | 0xC_    | 0xD_ | 0xE_ | 0xF_ | Byte 7 opcodes |
|------|---------|---------|--------|--------|---------|---------|--------|----------|---------|----------|----------|----------|---------|------|------|------|----------------|
| 0x_0 | NOP     | MACX,0  | RREG,0 | WREG,0 | LOOP    | HOP,RF  | COND,T | COND,0   | WREG,UI | PASS     | ROR      | IB0      | FORK,RF |      |      |      | CST,□          |
| 0x_1 | CLRL    | MACX,1  | RREG,1 | WREG,1 | LRET    | HOP,RR  | COND,F | COND,K   | WREG,B1 | NOT      | ROL      | IB1      | FORK,RR |      |      |      | CSF,□          |
| 0x_2 | MACX,L  | MACX,2  | RREG,2 | WREG,2 | LBRK    | HOP,RL  | COND,Z | COND,X   | WREG,B2 | INC      | RRX      | IB2      | FORK,RL |      |      |      | CAT,□          |
| 0x_3 | PVMAC   | MACX,3  | RREG,3 | WREG,3 | LRST    | HOP,RB  | COND,N | COND,R   | WREG,AC | DEC      | RLX      | IB3      | FORK,RB |      |      |      | CAF,□          |
| 0x_4 | GPMAC   | MACX,4  | RREG,4 | WREG,4 | RET     | HOP,AF  | COND,V | COND,A   | RREG,AC | NEG      | LSR      | ZB0      | FORK,AF |      |      |      | ROT,□          |
| 0x_5 | WREG,DP | MACX,5  | RREG,5 | WREG,5 | JRST    | HOP,AR  | COND,C | COND,B   | RREG,U0 | WSW      | LSL      | ZB1      | FORK,AR |      |      |      |                |
| 0x_6 | RREG,L  | MACX,6  | RREG,6 | WREG,6 | MREAD   | HOP,AL  | COND,M | COND,P   | RREG,B0 | BRR      | ASR      | ZB2      | FORK,AL |      |      |      |                |
| 0x_7 | WREG,L  | MACX,7  | RREG,7 | WREG,7 | MWRITE  | HOP,AB  | COND,W | COND,D   | RREG,Z  | BRL      | BSL      | ZB3      | FORK,AB |      |      |      |                |
| 0x_8 | END     | MACX,8  | RREG,8 | WREG,8 | RREG,MA | WREG,MA | CLRA   | RREG,FS0 | ACOFF   | BSET     | OR       | NOR      | FORK,MA |      |      |      |                |
| 0x_9 |         | MACX,9  | RREG,9 | WREG,9 | RREG,MX | WREG,MX | SETA   | RREG,FS1 | ACON    | BCLR     | XOR      | XNOR     | TERM    |      |      |      |                |
| 0x_A |         | MACX,10 | RREG,A | WREG,A | RREG,MY | WREG,MY | CLRB   | PVFLG    | ACLR    | BINV     | IMP      | INH      | DUPE    |      |      |      |                |
| 0x_B |         | MACX,11 | RREG,B | WREG,B | RREG,MP | WREG,MP | SETB   | GPFLG    |         | BTST     | CMP      | NAND     | TRMD    |      |      |      |                |
| 0x_C |         | MACX,12 | RREG,C | WREG,C | RSTMA   | REDO    | CLRFLT | CLRC     |         | ADD      | SUB      | AND      | RREG,AP |      |      |      |                |
| 0x_D | LDW***  | MACX,13 | RREG,D | WREG,D | JMP***  | JSR***  | SETFLT | SETC     |         | ADD,W*** | SUB,W*** | AND,W*** | RREG,EP | ***  | ***  | ***  |                |
| 0x_E | LDB**   | MACX,14 | RREG,E | WREG,E | PAGE**  |         | CLR**  | FSEL **  | **      | ADD,B1** | SUB,B1** | AND,B1** | HALT    | **   | **   | **   |                |
| 0x_F | WRMAC** | MACX,15 | RREG,F | WREG,F | PDEF**  |         | SET**  | COND**   | LSET**  | ADD,B2** | SUB,B2** | AND,B2** | RESET   | **   | **   | **   |                |

| LSET: ALU Loop Mode Bits  |                                          |
|---------------------------|------------------------------------------|
| Unary Input 1:            | b0-1                                     |
| Binary Input 1:           | b2-3                                     |
| Binary Input 2:           | b4-5                                     |
| Unimplemented:            | b6-7                                     |
|                           | 00=No Change                             |
|                           | 01=Unary ALU Out                         |
|                           | 10=Binary ALU Out                        |
|                           | 11=Clear Loop                            |
| Common settings:          |                                          |
| LSET 0x01:                | Loop Unary ALU Output to Unary Input     |
| LSET 0x02:                | Loop Binary ALU Output to Unary Input    |
| LSET 0x04:                | Loop Unary ALU Output to Binary Input 1  |
| LSET 0x08:                | Loop Binary ALU Output to Binary Input 1 |
| LSET 0x10:                | Loop Unary ALU Output to Binary Input 2  |
| LSET 0x20:                | Loop Binary ALU Output to Binary Input 2 |
| LSET 0x03:                | Clear Unary ALU Input 1 Loop             |
| LSET 0x0C:                | Clear Binary ALU Input 1 Loop            |
| LSET 0x30:                | Clear Binary ALU Input 2 Loop            |
| LSET 0x3F:                | Clear All ALU Input Loops                |
|                           |                                          |
| General Purpose Registers |                                          |
| 0-9                       | Private, per-process registers           |
| A-F                       | Shared between all processes             |
|                           |                                          |

| 7654 3210                                   | Instruction Bit Decoding |
|---------------------------------------------|--------------------------|
| 000- ----                                   | Instruction proc         |
| 001- ----                                   | Register Bank            |
| 010- ----                                   | Program counter & Memory |
| 011- ----                                   | Flag & Conditional       |
| 10-- ----                                   | ALU                      |
| 1100 ----                                   | Context Switching        |
| 1101 ----                                   | Game Subsystem 1         |
| 111- ----                                   | Game Subsystem 2         |
|                                             |                          |
| Defining a new page:                        |                          |
| Set registers MX,MY with coordinates        |                          |
| Use PAGE opcode to set page number          |                          |
| ('F' flag can be used to verify page use)   |                          |
| Use PDEF opcode with page size specifier to |                          |
| Create page.                                |                          |
|                                             |                          |
| JMP/JSR: use PAGE first to set page to jump |                          |
| to, or 'PAGE 0' for current page.           |                          |
| JMP/JSR/PDEF will clear PAGE register to 0. |                          |
|                                             |                          |
| Page size specs:                            |                          |
| 0                                           | clear page               |
| 1                                           | 256x256                  |
| 2                                           | 128x128                  |
| 3                                           | 64x64                    |
| 4                                           | 32x32                    |
| 5                                           | 16x16                    |

| Conditional Byte (Inst. word Byte 3/7)                         |                       |                       |       |          |
|----------------------------------------------------------------|-----------------------|-----------------------|-------|----------|
| Instruction word:<br>0 = 1 opcode byte<br>1 = 3/7 opcode bytes |                       | Bit 7                 | Bit 6 | Bit 5    |
| Flag: 0=auto sel., 1=selected                                  |                       |                       |       | Bit 4    |
| Compare against:0=false, 1=true                                |                       |                       |       | Bits 0-3 |
| Breakpoint set:0=false, 1=true                                 |                       |                       |       |          |
| Action:<br>(squ grid)                                          | Action:<br>(hex grid) | Action:<br>(tri grid) |       |          |
| 0000=NOP                                                       | 0000=NOP              | 0000=NOP              |       |          |
| 0001=ROT,RR                                                    | 0001=ROT,RR1          | 0001=ROT,RR           |       |          |
| 0010=ROT,RL                                                    | 0010=ROT,RL1          | 0010=ROT,RL           |       |          |
| 0011=ROT,RB                                                    | 0011=ROT,RB           | 0011=ROT,RB           |       |          |
| 0100=ROT,AF                                                    | 0100=ROT,RR2          | 0100=ROT,PF           |       |          |
| 0101=ROT,AR                                                    | 0101=ROT,RL2          | 0101=ROT,SL           |       |          |
| 0110=ROT,AL                                                    | 0110=ROT,AL           | 0110=ROT,SR           |       |          |
| 0111=ROT,AB                                                    | 0111=ROT,AR           | 0111=ROT,PB           |       |          |
| 1000=REDO                                                      | 1000=REDO             | 1000=REDO             |       |          |
| 1001=TERM                                                      | 1001=TERM             | 1001=TERM             |       |          |
| 1010=LRET                                                      | 1010=LRET             | 1010=LRET             |       |          |
| 1011=LBRK                                                      | 1011=LBRK             | 1011=LBRK             |       |          |
| 1100=RET                                                       | 1100=RET              | 1100=RET              |       |          |
| 1101=                                                          | 1101=                 | 1101=                 |       |          |
| 1110=                                                          | 1110=                 | 1110=                 |       |          |
| 1111=                                                          | 1111=                 | 1111=                 |       |          |

| Base CPU Flag IDs: |          |         |                                     |
|--------------------|----------|---------|-------------------------------------|
| Numeric            | Mnemonic | FS0 Bit | Description                         |
| 0                  | T        | b0      | Always True                         |
| 1                  | F        | b1      | Fault                               |
| 2                  | Z        | b2      | Zero                                |
| 3                  | N        | b3      | Negative                            |
| 4                  | V        | b4      | Overflow                            |
| 5                  | C        | b5      | Carry                               |
| 6                  | A        | b6      | Private A Flag                      |
| 7                  | B        | b7      | Private B Flag                      |
| 8                  | M        | b8      | Memory Address Fault                |
| 9                  | W        | b9      | Memory Write Fault                  |
| 10                 | O        | b10     | Memory Cell Owner                   |
| 11                 | S        | b11     | Strong Fault                        |
| 12                 | -        | b12     | Unused                              |
| 13                 | R        | b10     | Random Bit                          |
| 14                 | P        | b11     | Positive (NOR of 'N' and 'V' flags) |
| 15                 | D        | b12     | Temp Register Zero                  |
| 16-31              | G0-G15   | b16-31  | Group 0-15 (shared or private)      |
| 32-63              | -        | b32-63  | Reserved for expansion              |

| Read/Write                           | Register List                            | Read-Only                      |
|--------------------------------------|------------------------------------------|--------------------------------|
| Register L – LOAD Register           |                                          | Register Z – Static 0x00000000 |
| Register MA – Macro Address Register |                                          |                                |
|                                      | Register XY – Memory XY Rel. Address     |                                |
| Register 0 – General Purpose 0       | Register AP – Active/Dupe Process Counts |                                |
| Register 1 – General Purpose 1       | Register EP – Extended Process Info.     |                                |
| Register 2 – General Purpose 2       |                                          |                                |
| Register 3 – General Purpose 3       | Register UO – ALU_UO Output Register     |                                |
| Register 4 – General Purpose 4       | Register AO – ALU_BO Output Register     |                                |
| Register 5 – General Purpose 5       |                                          | Write-Only                     |
| Register 6 – General Purpose 6       | Register DP – Display Register           |                                |
| Register 7 – General Purpose 7       |                                          |                                |
| Register 8 – General Purpose 8       | Register UI – ALU_UI Input Register      |                                |
| Register 9 – General Purpose 9       | Register B1 – ALU_B1 Input Register      |                                |
| Register A – Group Register A        | Register B2 – ALU_B2 Input Register      |                                |
|                                      |                                          |                                |
| Register MD – Memory Data Access     |                                          |                                |
| Register PC – Program Counter        |                                          |                                |
|                                      |                                          |                                |
| Register AC – ALU Config Register    |                                          |                                |

| Process Control Block Memory (Per-Process) |
|--------------------------------------------|
| Default 64-bits each unless indicated      |
| 0001 – LOAD Register                       |
| 0001 – Macro Addr (1B)                     |
| 0001 – Selected Macro Group (1B)           |
| 0001 – Macro 0                             |
| 0001 – Macro 1                             |
| 0001 – Macro 2                             |
| 0001 – Macro 3                             |
| 0001 – Macro 4                             |
| 0001 – Macro 5                             |
| 0001 – Macro 6                             |
| 0001 – Macro 7                             |
| 0001 – Macro 8                             |
| 0001 – Macro 9                             |
| 0001 – Macro 10                            |
| 0001 – Macro 11                            |
| 0001 – Macro 12                            |
| 0001 – Macro 13                            |
| 0001 – Macro 14                            |
| 0001 – Macro 15                            |
| 0001 – Display Register                    |
| 0001 – Temp. Register 1                    |
| 0010 – General Purpose 0                   |
| 0010 – General Purpose 1                   |
| 0010 – General Purpose 2                   |
| 0010 – General Purpose 3                   |
| 0010 – General Purpose 4                   |
| 0010 – General Purpose 5                   |
| 0010 – General Purpose 6                   |
| 0010 – General Purpose 7                   |
| 0010 – General Purpose 8                   |
| 0010 – General Purpose 9                   |
| 0010 – Temp. Register 2                    |
| 0010 – Pvt./Grp. Config Bitmap (2B)        |
| 0110 – Program Counter (3B)                |
| 0110 – JSR Return Address Stack (13B)      |
| 0110 – Loop Return Address Stack (13B)     |
| 0110 – Memory Data Access                  |
| 0110 – Memory XY Rel. Address (2B)         |
| 0110 – Temp. Register 6                    |
| 0011 – Flag Storage                        |
| 0011 – Flag Select S + A (2B)              |
| 0011 – Temp. Register 3                    |
| 0100 – ALU_UI Input Register               |
| 0100 – ALU_UO Output Register              |
| 0100 – ALU_B1 Input Register               |
| 0100 – ALU_B2 Input Register               |
| 0100 – ALU_BO Output Register              |
| 0100 – ALU Config Register (3B)            |
| 0100 – Temp. Register 4                    |
| 0101 – Proc. Status & Dupe Count           |
| xxxx – Dummy (1B)                          |

| Module Network Addresses                |
|-----------------------------------------|
| 0001 = Instruction Processor            |
| 0010 = Register Bank                    |
| 0011 = Conditional & Flag Subsystem     |
| 0100 = ALU Subsystem                    |
| 0101 = Context Switcher                 |
| 0110 = Program Counter Control & Memory |
| 0111 = Game Feature Subsystem           |
| 1000 =                                  |
| 1001 =                                  |
| 1010 =                                  |
| 1011 =                                  |
| 1100 =                                  |
| 1101 =                                  |
| 1110 =                                  |
| 1111 =                                  |

|                                |            |                         |                                    |                                    |
|--------------------------------|------------|-------------------------|------------------------------------|------------------------------------|
|                                | Bits 19-23 | Bits 16-18              | Bits 8-15                          | Bits 0-7                           |
| Register PC<br>Program Counter | Zeroes     | PC Direction<br>0-3/0-5 | Program Counter Y<br>Address 0-255 | Program Counter X<br>Address 0-255 |

|                                          |            |                           |            |                           |
|------------------------------------------|------------|---------------------------|------------|---------------------------|
|                                          | Bits 26-31 | Bits 16-25                | Bits 10-15 | Bits 0-9                  |
| Register 0x1D<br>Active Process<br>Count | Zeroes     | Maximum UID Number 1-1023 | Zeroes     | User Process Count 1-1023 |

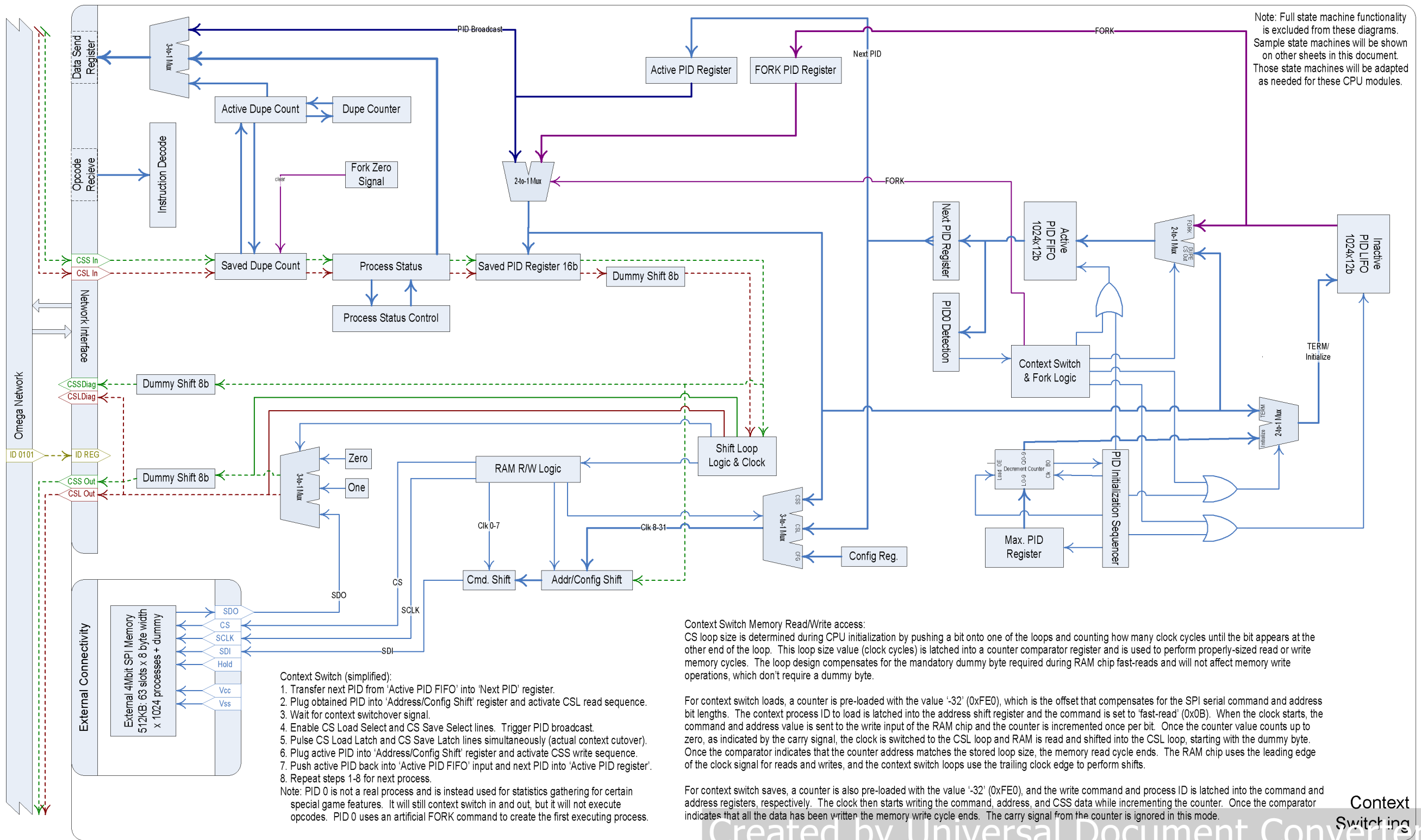
|                                 | Bits 24-31                                    | Bits 8-23                                             | Bit 7 | Bit 6 | Bit 5 | Bit 4 | Bit 3 | Bit 2 | Bit 1 | Bit 0 |
|---------------------------------|-----------------------------------------------|-------------------------------------------------------|-------|-------|-------|-------|-------|-------|-------|-------|
| PCB<br>Extended<br>Process Info | Process duplicate<br>Count 0-255<br>(primary) | Empty                                                 |       |       |       |       |       |       |       |       |
|                                 |                                               | unimplemented                                         |       |       |       |       |       |       |       |       |
|                                 |                                               | unimplemented                                         |       |       |       |       |       |       |       |       |
|                                 |                                               | unimplemented                                         |       |       |       |       |       |       |       |       |
|                                 |                                               | unimplemented                                         |       |       |       |       |       |       |       |       |
|                                 |                                               | 0 = off, 1 = Process has underlay addressing enabled. |       |       |       |       |       |       |       |       |
|                                 |                                               | 0 = off, 1 = Process is an underlay process.          |       |       |       |       |       |       |       |       |
|                                 |                                               | 0 = off, 1 = Process is currently strong-faulted.     |       |       |       |       |       |       |       |       |
|                                 |                                               | 0 = off, 1 = Process is terminated.                   |       |       |       |       |       |       |       |       |

|                                                                 |                            |           |       |       |                       |                      |                           |                                                                     |                                                           |                 |
|-----------------------------------------------------------------|----------------------------|-----------|-------|-------|-----------------------|----------------------|---------------------------|---------------------------------------------------------------------|-----------------------------------------------------------|-----------------|
|                                                                 | Bits 16-31                 | Bits 8-15 | Bit 7 | Bit 6 | Bit 5                 | Bit 4                | Bit 3                     | Bit 2                                                               | Bit 1                                                     | Bit 0           |
| Register<br>Aux. Memory<br>Data Register<br>(no longer visible) | Occupation Markers<br>0-15 | Zeroes    |       |       | 00 =<br>Unowned Cells | Special<br>Block T/F | TERM on PC<br>Contact T/F | MAR contact:<br>00=Normal<br>01=F.Fault<br>10=S.Fault<br>11=Barrier |                                                           |                 |
|                                                                 |                            |           |       |       |                       |                      |                           |                                                                     | 01 = write enable<br>10 = interface<br>11 = write protect | Cell Owner 0-15 |

|                                       |
|---------------------------------------|
| b0-b3 = character                     |
| b4 = U0 highlight color or background |
| b5 = U1 highlight color or background |
| b6 = U2 highlight color or background |
| b7 = U3 highlight color or background |
|                                       |
| 3b char color:                        |
| 000      background (black or user)   |
| 001      foreground (gray)            |
| 010      random color 1               |
| 011      random color 2               |
| 100      U0 selective color (red)     |
| 101      U1 selective color (green)   |
| 110      U2 selective color (blue)    |
| 111      U3 selective color (yellow)  |

|                     |                     |
|---------------------|---------------------|
| 0000 = Empty Cell   | 0100 = Mined        |
| 0001 = Glitch Cell  | 0101 = Trapped      |
| 0010 = Corrupt Cell | 0110 = Corrupt Cell |
| 0011 = Memory Hole  | 0111 = N/A          |

|                                                                                                           |
|-----------------------------------------------------------------------------------------------------------|
| ALU configuration (Use RREG,AC & WREG,AC to read and update)<br>Default configuration is all zero values. |
| Byte 0: Basic ALU and auto-output configuration                                                           |
| <b>b0-1: ALU configuration update method:</b>                                                             |
| 00=write: write 30-bit input value to ALU config register                                                 |
| 01=OR: for each input bit set, set corresponding config bit                                               |
| 10=INH: for each input bit set, clear corresponding config bit                                            |
| 11=XOR: for each input bit set, toggle corresponding config bit                                           |
| <b>b2-7: <i>unimplemented</i></b>                                                                         |
|                                                                                                           |
| <b>Byte 1: Flag configuration</b>                                                                         |
| <b>b8-9: C/V flag source:</b> 00=auto,01=ALU_U0,10=ALU_B0,11=ignore                                       |
| <b>b10-11: PNZ flag source:</b> 00=auto,01=ALU_U0,10=ALU_B0,11=ignore                                     |
| <b>b12-13: ALU_U auto flag select:</b> 00=Z,01=C,10=none,11=V                                             |
| <b>b14-15: ALU_B auto flag select:</b> 00=Z,01=C,10=none,11=V                                             |
|                                                                                                           |
| Byte 2: ALU input/crossover/auto-output configuration                                                     |
| <b>b16-17: ALU_UI Auto-in:</b> 00=none,01=ALU_U0,10=ALU_B0,11=Active-Out                                  |
| <b>b18-19: ALU_B1 Auto-in:</b> 00=none,01=ALU_U0,10=ALU_B0,11=Active-Out                                  |
| <b>b20-21: ALU_B2 Auto-in:</b> 00=none,01=ALU_U0,10=ALU_B0,11=Active-Out                                  |
| <b>b22: Disable ALU_U0 auto-output</b> T/F                                                                |
| <b>b23: Disable ALU_B0 auto-output</b> T/F                                                                |



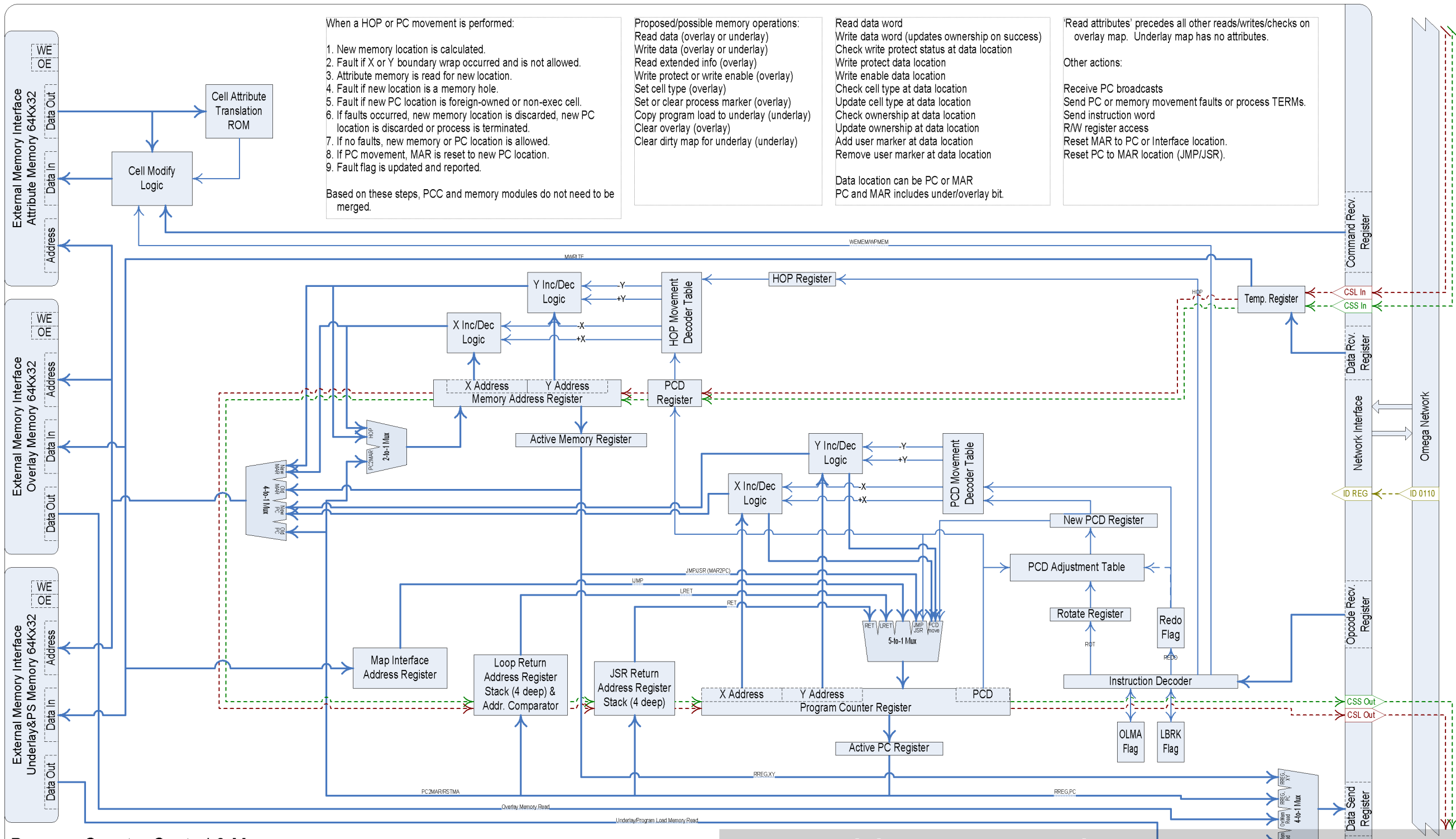
Context Switch Memory Read/Write access:  
CS loop size is determined during CPU initialization by pushing a bit onto one of the loops and counting how many clock cycles until the bit appears at the other end of the loop. This loop size value (clock cycles) is latched into a counter comparator register and is used to perform properly-sized read or write memory cycles. The loop design compensates for the mandatory dummy byte required during RAM chip fast-reads and will not affect memory write operations, which don't require a dummy byte.

For context switch loads, a counter is pre-loaded with the value '-32' (0xFE0), which is the offset that compensates for the SPI serial command and address bit lengths. The context process ID to load is latched into the address shift register and the command is set to 'fast-read' (0x0B). When the clock starts, the command and address value is sent to the write input of the RAM chip and the counter is incremented once per bit. Once the counter value counts up to zero, as indicated by the carry signal, the clock is switched to the CSL loop and RAM is read and shifted into the CSL loop, starting with the dummy byte. Once the comparator indicates that the counter address matches the stored loop size, the memory read cycle ends. The RAM chip uses the leading edge of the clock signal for reads and writes, and the context switch loops use the trailing clock edge to perform shifts.

For context switch saves, a counter is also pre-loaded with the value '-32' (0xFE0), and the write command and process ID is latched into the command and address registers, respectively. The clock then starts writing the command, address, and CSS data while incrementing the counter. Once the comparator indicates that all the data has been written the memory write cycle ends. The carry signal from the counter is ignored in this mode.

Context Switching





When a HOP or PC movement is performed:

1. New memory location is calculated.
2. Fault if X or Y boundary wrap occurred and is not allowed.
3. Attribute memory is read for new location.
4. Fault if new location is a memory hole.
5. Fault if new PC location is foreign-owned or non-exec cell.
6. If faults occurred, new memory location is discarded, new PC location is discarded or process is terminated.
7. If no faults, new memory or PC location is allowed.
8. If PC movement, MAR is reset to new PC location.
9. Fault flag is updated and reported.

Based on these steps, PCC and memory modules do not need to be merged.

Proposed/possible memory operations:

- Read data (overlay or underlay)
- Write data (overlay or underlay)
- Read extended info (overlay)
- Write protect or write enable (overlay)
- Set cell type (overlay)
- Set or clear process marker (overlay)
- Copy program load to underlay (underlay)
- Clear overlay (overlay)
- Clear dirty map for underlay (underlay)

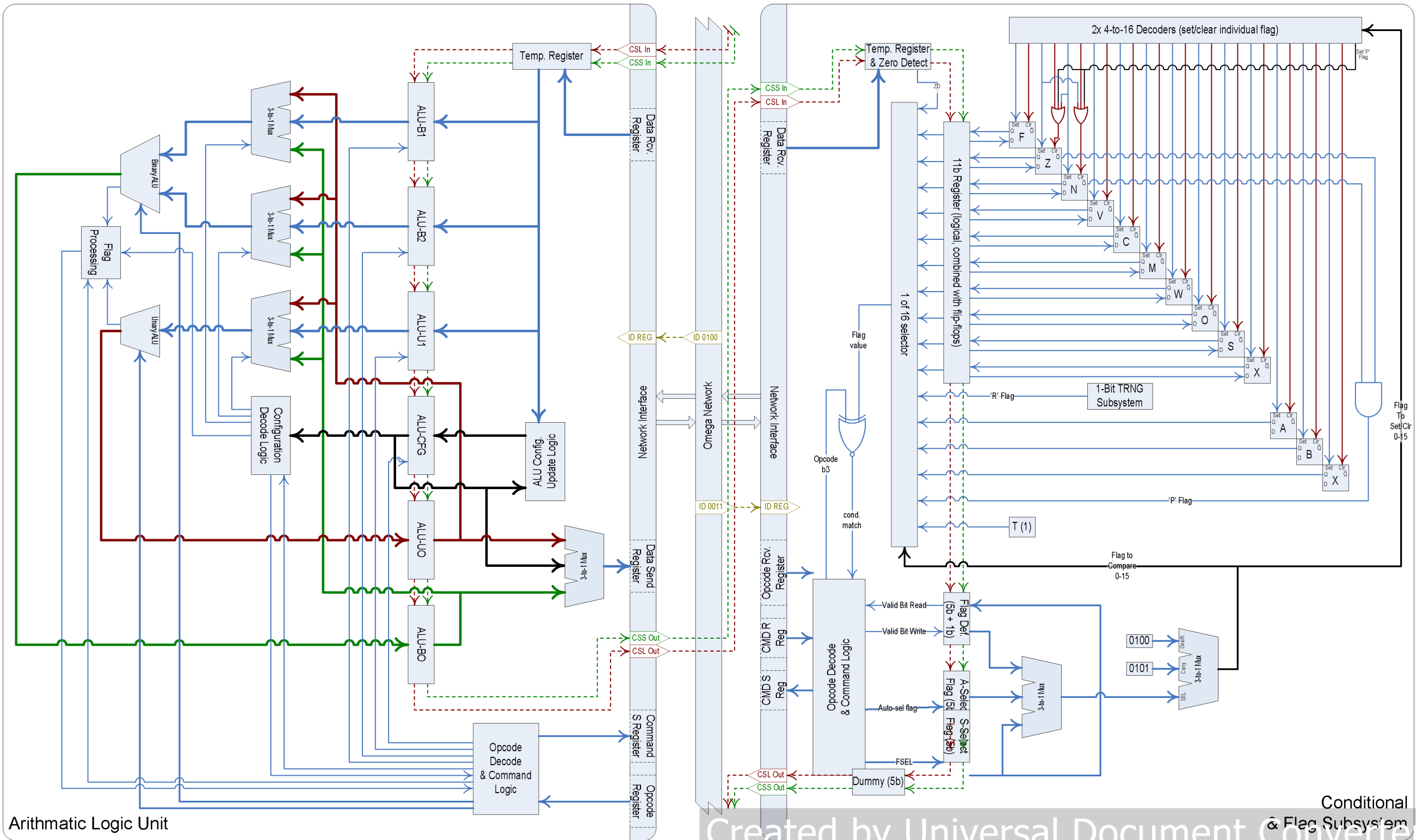
Read data word  
Write data word (updates ownership on success)  
Check write protect status at data location  
Write protect data location  
Write enable data location  
Check cell type at data location  
Update cell type at data location  
Check ownership at data location  
Update ownership at data location  
Add user marker at data location  
Remove user marker at data location

Data location can be PC or MAR  
PC and MAR includes under/overlay bit.

'Read attributes' precedes all other reads/writes/checks on overlay map. Underlay map has no attributes.

Other actions:

- Receive PC broadcasts
- Send PC or memory movement faults or process TERMS.
- Send instruction word
- R/W register access
- Reset MAR to PC or Interface location.
- Reset PC to MAR location (JMP/JSR).



Macro Execution Example

64-bit instruction word: 

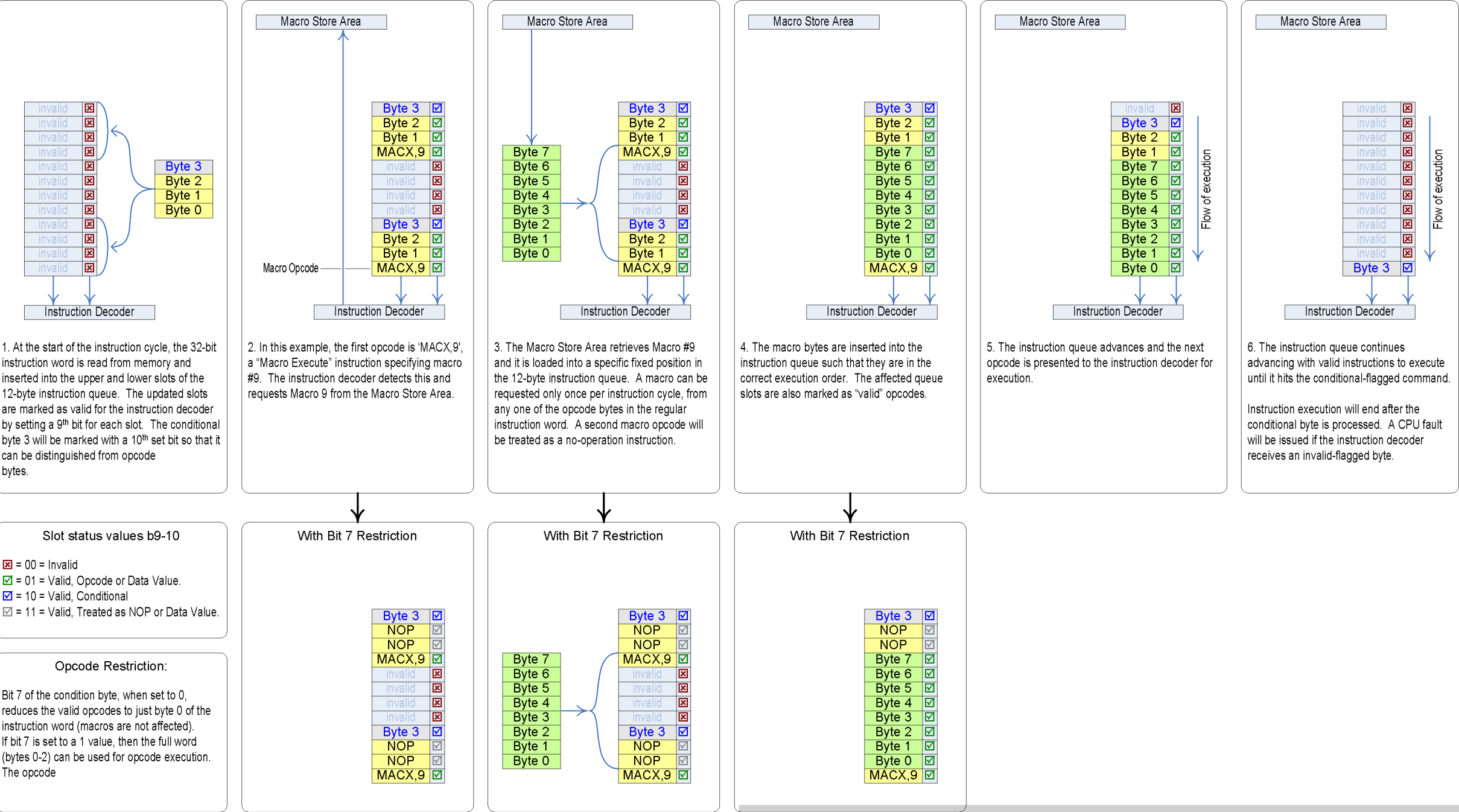
|        |        |        |        |
|--------|--------|--------|--------|
| Byte 3 | Byte 2 | Byte 1 | Byte 0 |
|--------|--------|--------|--------|

  
Bytes 0-2 are regular opcode commands, byte 3 is an ending conditional instruction.

64-bit macro: 

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| Byte 7 | Byte 6 | Byte 5 | Byte 4 | Byte 3 | Byte 2 | Byte 1 | Byte 0 |
|--------|--------|--------|--------|--------|--------|--------|--------|

  
Bytes 0-7 are regular opcode commands.



Macro Execution Example

64-bit instruction word: 

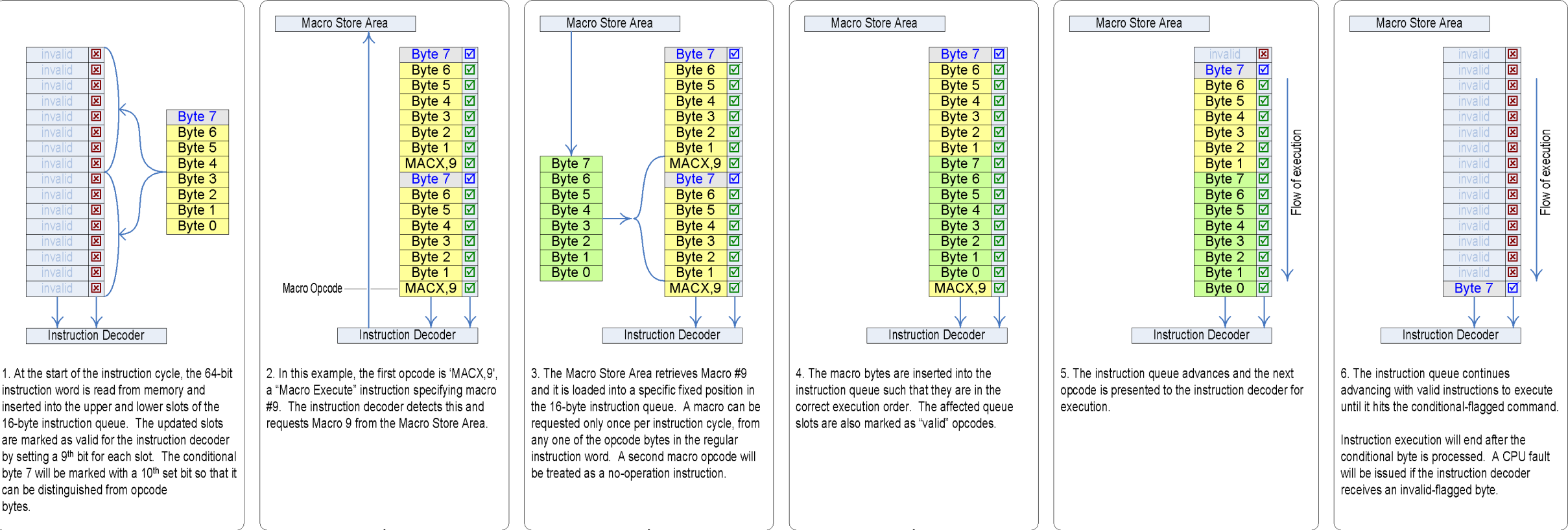
|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| Byte 7 | Byte 6 | Byte 5 | Byte 4 | Byte 3 | Byte 2 | Byte 1 | Byte 0 |
|--------|--------|--------|--------|--------|--------|--------|--------|

  
Bytes 0-6 are regular opcode commands, byte 7 is an ending conditional instruction.

64-bit macro: 

|        |        |        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|--------|--------|
| Byte 7 | Byte 6 | Byte 5 | Byte 4 | Byte 3 | Byte 2 | Byte 1 | Byte 0 |
|--------|--------|--------|--------|--------|--------|--------|--------|

  
Bytes 0-7 are regular opcode commands.

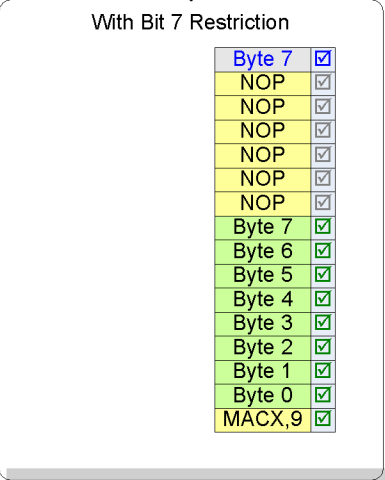
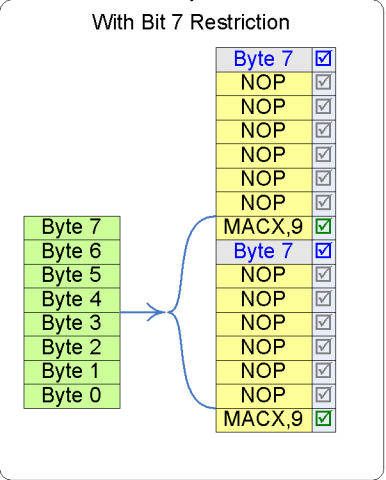
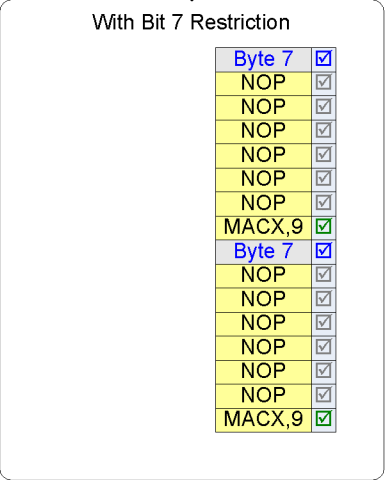


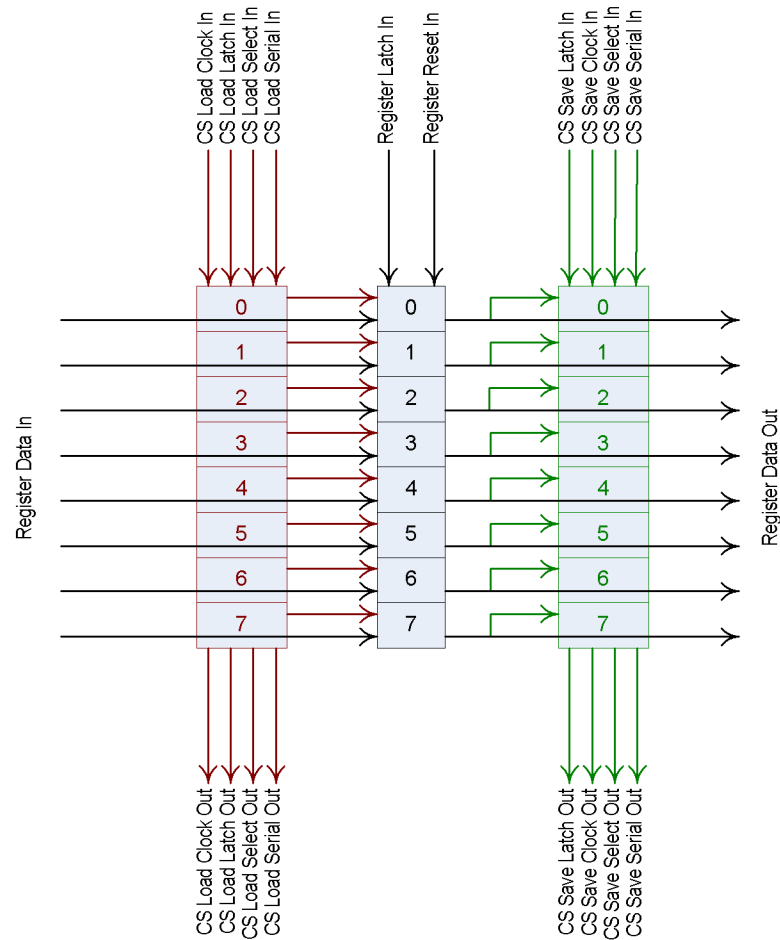
Slot status values b9-10

|         |                             |                    |                                     |
|---------|-----------------------------|--------------------|-------------------------------------|
| Invalid | Valid, Opcode or Data Value | Valid, Conditional | Valid, Treated as NOP or Data Value |
|---------|-----------------------------|--------------------|-------------------------------------|

Opcode Restriction:

Bit 7 of the condition byte, when set to 0, reduces the valid opcodes to just byte 0 of the instruction word (macros are not affected). If bit 7 is set to a 1 value, then the full word (bytes 0-6) can be used for opcode execution. The opcode





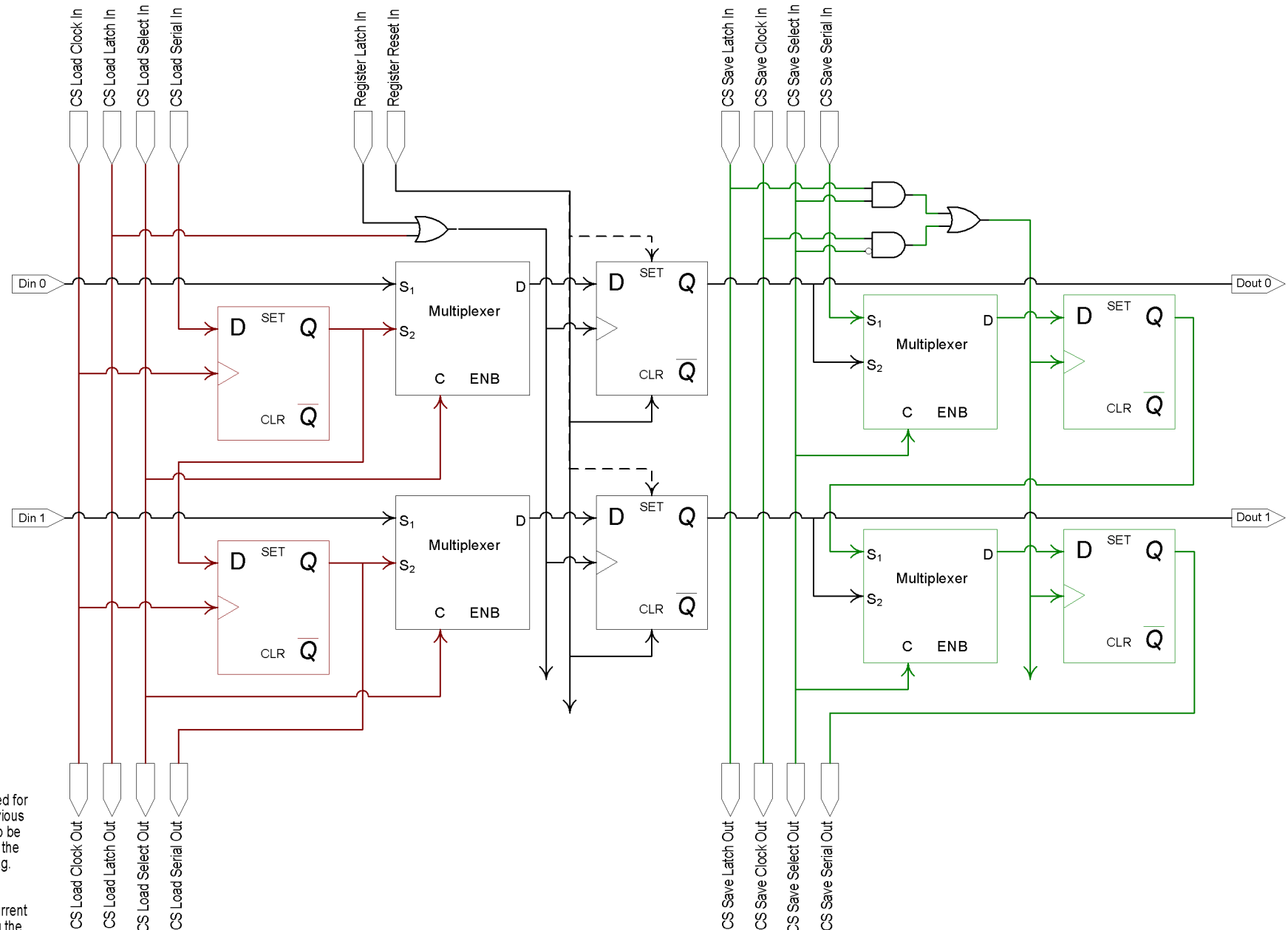
This register concept is used for "saveable" CPU registers that need to be stored and loaded during the context switching of processes.

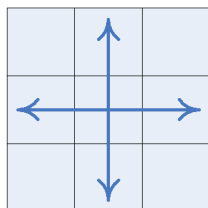
The register consists of a regular parallel I/O register attached to two serial shift registers – the first one (in red) used for readying the next process context to be loaded in, and the second one (in green) to be used for saving out the previous process context. The dual latch lines connected to the parallel register direct multiplexers to select which input is to be latched, and the CS latch lines carry out the actual latching of data into the register. The CS clock lines are pulsed the exact number of data shifts needed to transfer the serial data and only operate when actual data shifting is occurring. The speed of these clock pulses is based on the timing requirements of the process control block (PCB) memory.

For this register, a context switch is performed by bringing the CS Load/Save Select lines high, then latching the current parallel register into the "save" context switch (green) using the CS\_Save\_Latch signal and simultaneously latching the new processor data from the "load" context switch (red) into the parallel register using the CS\_Load\_Latch signal. While the new process is active, the context data from the now inactive process can then be shifted out and stored at the same time as the next process context is read and shifted into the "load" context switch. Part of this context switch functionality is also used for duplicating the register image of processes being forked.

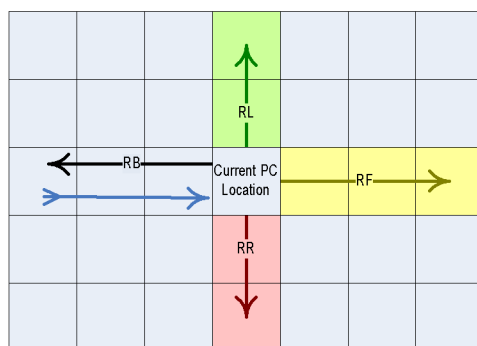
This approach minimizes the idle time of the CPU waiting for the context switch to complete.

Shown above: 8-bit simplified block diagram on the left, and a 2-bit "slice" of a standard logic diagram on the right.



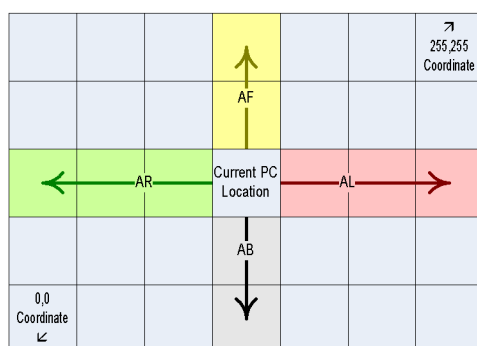


Execution Directions



Relative Directions

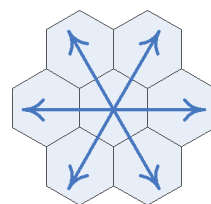
Blue Arrow: Current execution direction  
Yellow Arrow: Relative Forward (RF)  
Green Arrow: Relative Left (RL)  
Red Arrow: Relative Right (RR)  
Black Arrow: Relative Back (RB)



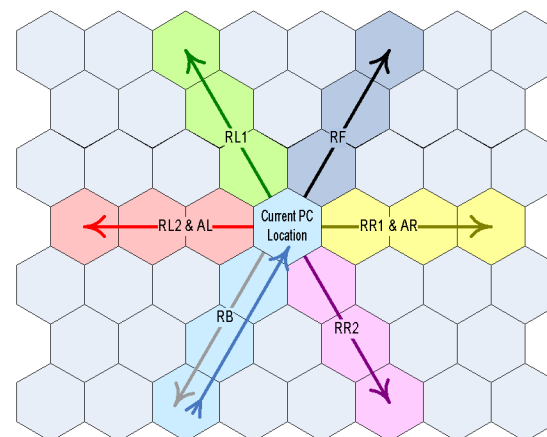
Absolute Directions

Yellow Arrow: Absolute Forward (AF)  
Black Arrow: Absolute Back (AB)  
Green Arrow: Absolute Right (AR)  
Red Arrow: Absolute Left (AL)

## Square Grid

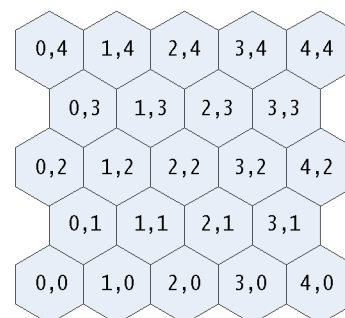


Execution Directions

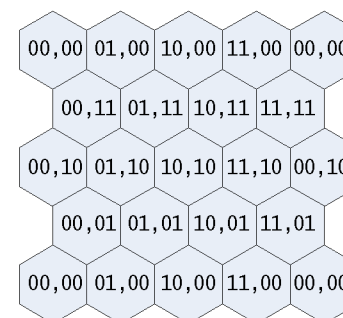


Relative and Absolute Directions

Blue Arrow: Current execution direction  
Black Arrow: Relative Forward (RF)  
Gray Arrow: Relative Backward (RB)  
Yellow Arrow: Relative Right 1 (RR1) and Absolute Right (AR)  
Purple Arrow: Relative Right 2 (RR2)  
Green Arrow: Relative Left 1 (RL1)  
Red Arrow: Relative Left 2 (RL2) and Absolute Left (AL)

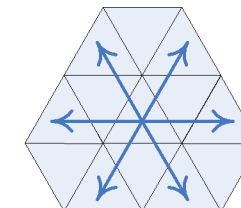


Coordinate System

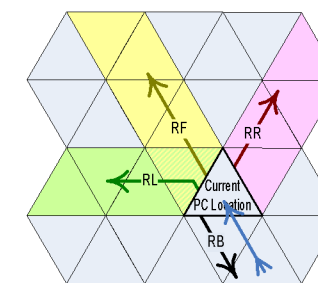


Coordinate System (Binary)

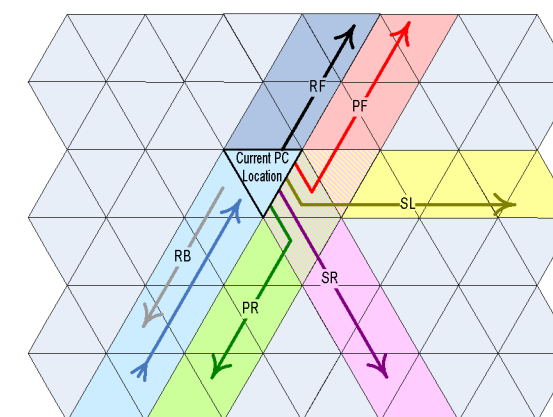
## Hexagonal Grid



Execution Directions

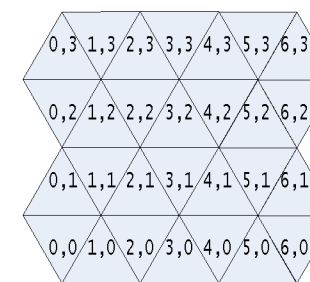


Relative Directions

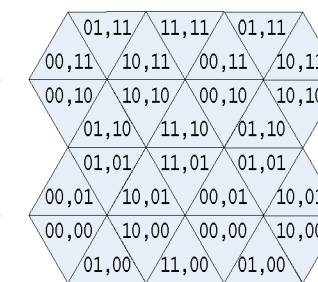


Parallel & Side Directions

Blue Arrow: Current execution direction  
Black Arrow: Relative Forward (RF)  
Gray Arrow: Relative Backward (RB)  
Gold Arrow: Side Left (SL)  
Purple Arrow: Side Right (SR)  
Red Arrow: Parallel Forward (PF)  
Green Arrow: Parallel Back (PB)

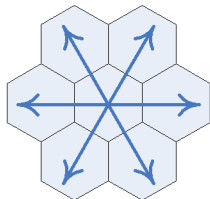


Coordinate System

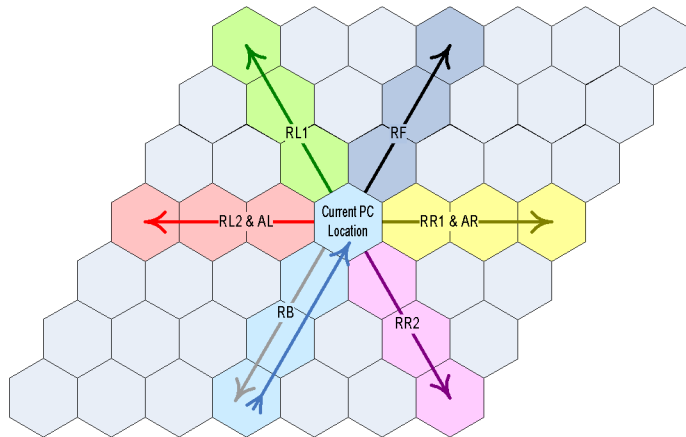


Coordinate System (Binary)

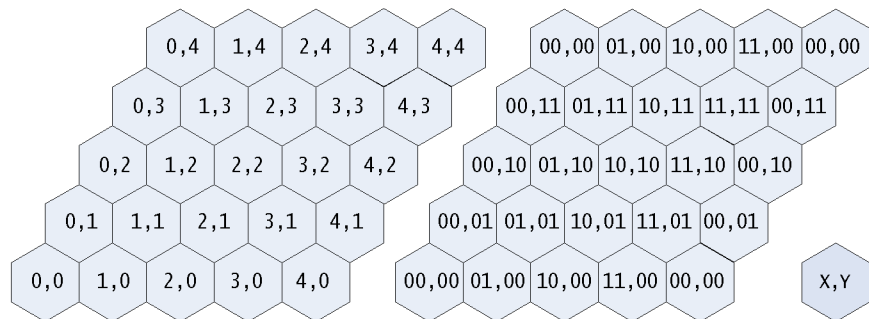
## Triangle Grid



Execution Directions



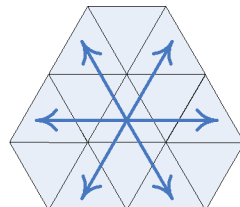
Relative and Absolute Directions  
Blue Arrow: Current execution direction  
Black Arrow: Relative Forward (RF)  
Gray Arrow: Relative Backward (RB)  
Yellow Arrow: Relative Right 1 (RR1) and Absolute Right (AR)  
Purple Arrow: Relative Right 2 (RR2)  
Green Arrow: Relative Left 1 (RL1)  
Red Arrow: Relative Left 2 (RL2) and Absolute Left (AL)



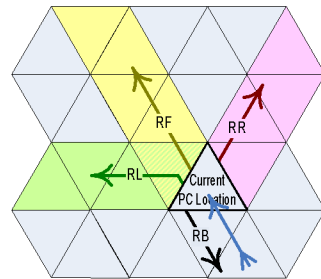
Coordinate System

Coordinate System (Binary 2LSB)

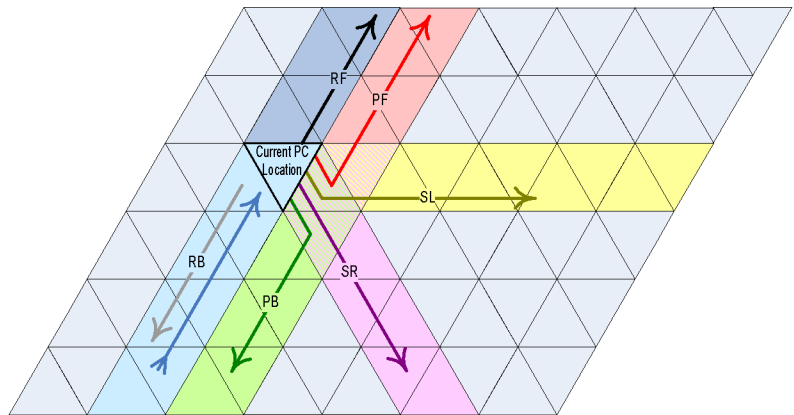
Hexagonal Grid (Alternate)



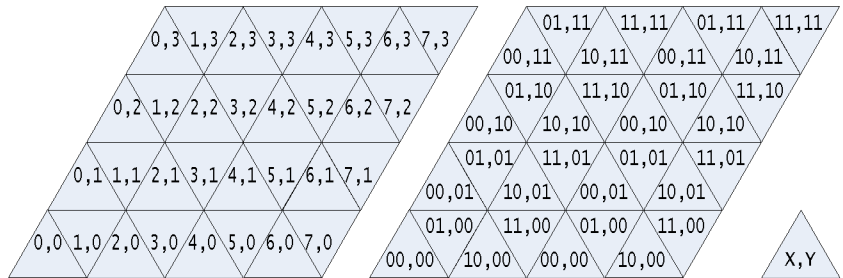
Execution Directions



Relative Directions



Parallel & Side Directions  
Blue Arrow: Current execution direction  
Black Arrow: Relative Forward (RF)  
Gray Arrow: Relative Backward (RB)  
Gold Arrow: Side Left (SL)  
Purple Arrow: Side Right (SR)  
Red Arrow: Parallel Forward (PF)  
Green Arrow: Parallel Back (PB)



Coordinate System

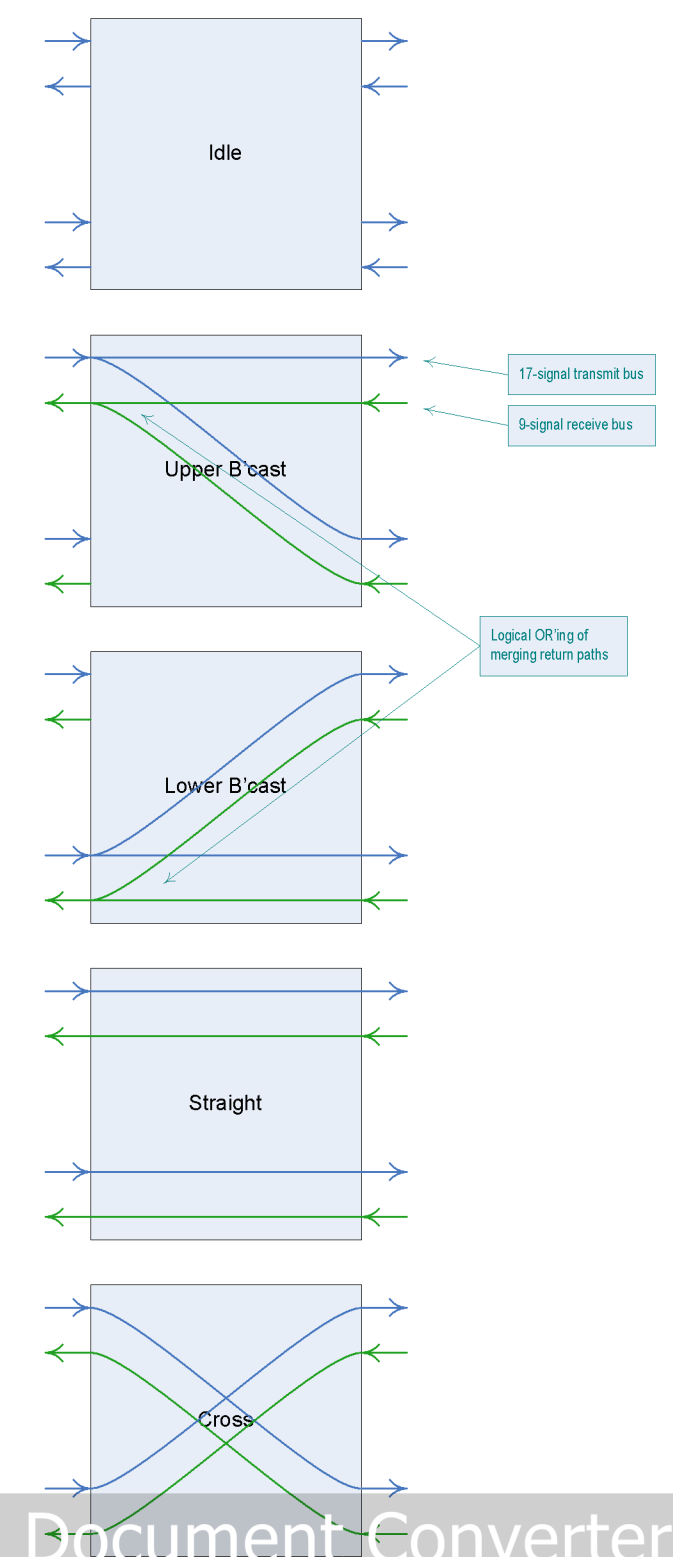
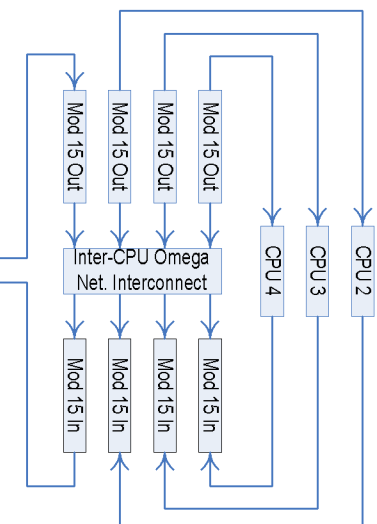
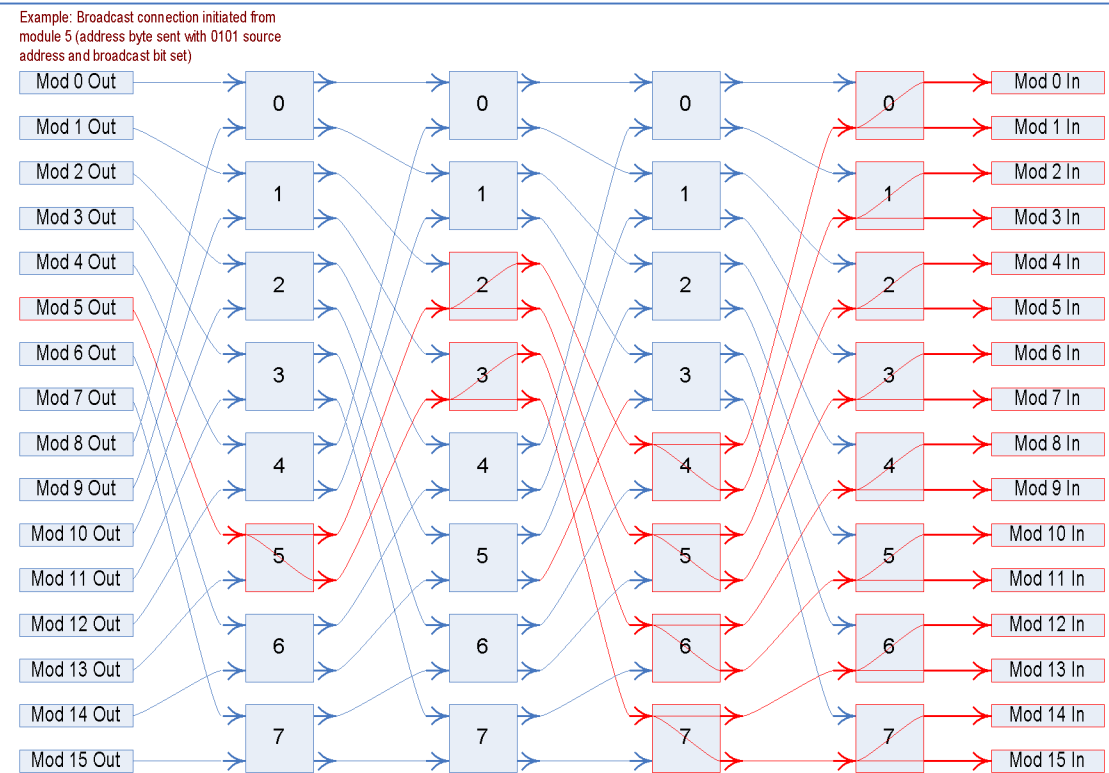
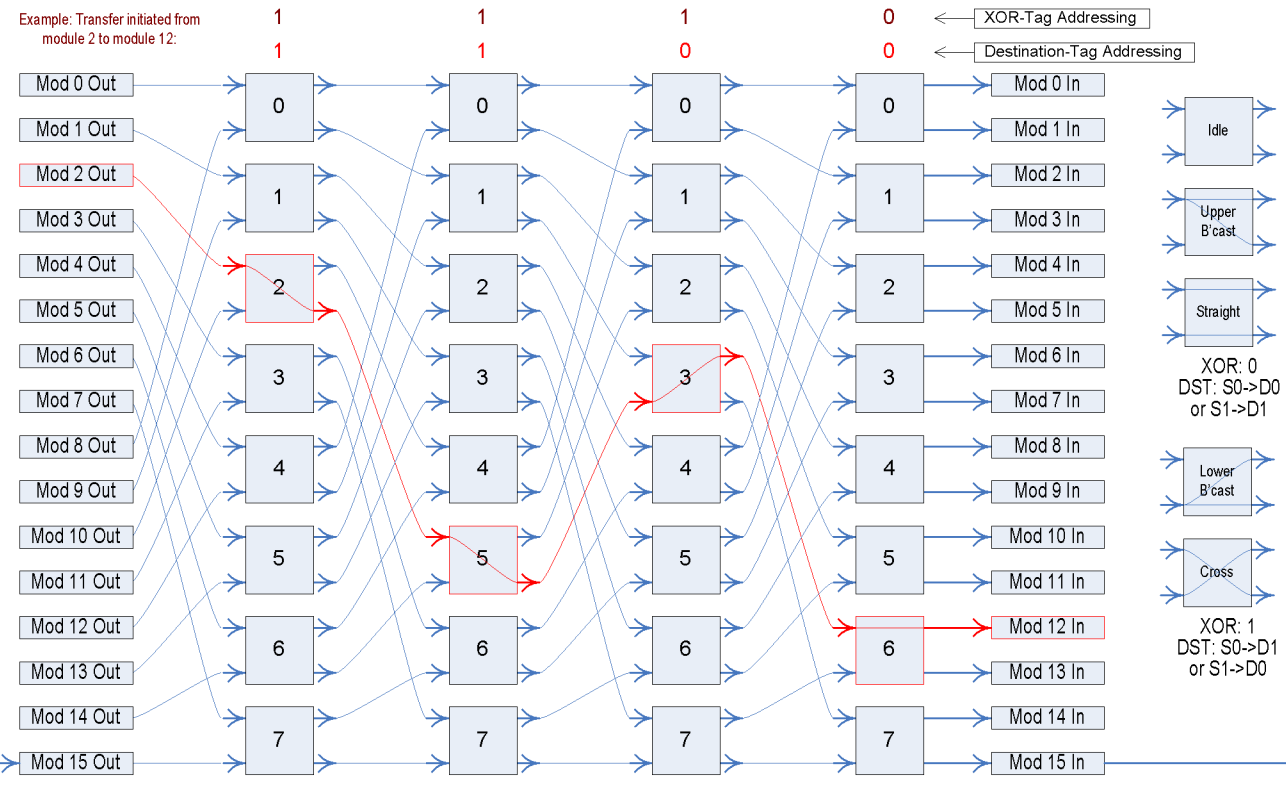
Coordinate System (Binary 2LSB)

Triangle Grid (Alternate)

| Hexagonal Grid | 0xB_      | 0xC_     | 0xD_     |
|----------------|-----------|----------|----------|
| 0x_0           | FORK, RF  | ROT, RF  | HOP, RF  |
| 0x_1           | FORK, RR  | ROT, RR  | HOP, RR  |
| 0x_2           | FORK, RR2 | ROT, RR2 | HOP, RR2 |
| 0x_3           | FORK, RL  | ROT, RL  | HOP, RL  |
| 0x_4           | FORK, RL2 | ROT, RL2 | HOP, RL2 |
| 0x_5           | FORK, RB  | ROT, RB  | HOP, RB  |
| 0x_6           | FORK, AR  | ROT, AR  | HOP, AR  |
| 0x_7           | FORK, AL  | ROT, AL  | HOP, AL  |

| Triangle Grid | 0xB_     | 0xC_    | 0xD_    |
|---------------|----------|---------|---------|
| 0x_0          | FORK, RF | ROT, RF | HOP, RF |
| 0x_1          | FORK, RR | ROT, RR | HOP, RR |
| 0x_2          | FORK, RL | ROT, RL | HOP, RL |
| 0x_3          | FORK, RB | ROT, RB | HOP, RB |
| 0x_4          | FORK, PF | ROT, PF | HOP, PF |
| 0x_5          | FORK, SR | ROT, SR | HOP, SR |
| 0x_6          | FORK, SL | ROT, SL | HOP, SL |
| 0x_7          | FORK, PB | ROT, PB | HOP, PB |

| Square Grid | 0xB_     | 0xC_    | 0xD_    |
|-------------|----------|---------|---------|
| 0x_0        | FORK, RF | ROT, RF | HOP, RF |
| 0x_1        | FORK, RR | ROT, RR | HOP, RR |
| 0x_2        | FORK, RL | ROT, RL | HOP, RL |
| 0x_3        | FORK, RB | ROT, RB | HOP, RB |
| 0x_4        | FORK, AF | ROT, AF | HOP, AF |
| 0x_5        | FORK, AR | ROT, AR | HOP, AR |
| 0x_6        | FORK, AL | ROT, AL | HOP, AL |
| 0x_7        | FORK, AB | ROT, AB | HOP, AB |



| Module Addresses                        | Flag Update Byte Order                                                       | Universal Module Commands                                    | Module-Specific Commands                                       |
|-----------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------|
| 0001 = Instruction Processor            | For full flag update:     order = FS3, FS2, FS1, FS0*                        | 00000000   INIT     initialization                           | 10000***   COND,*   conditional opcode (0-7)     Conditional   |
| 0010 = Register Bank                    | For ¾ flag update:       order = FS2, FS1, FS0*                              | 00000001   EOI     end of instruction                        | 10000001   PCXY     request PC XY             Memory           |
| 0011 = Conditional & Flag Subsystem     | For ½ update:           order = FS1, FS0*                                    | 00000010   BTI     broadcast data transfer to temp register. | 10000010   PCD     request PC direction fields     Memory      |
| 0100 = ALU Subsystem                    | For ¼ update:           order = FS0*                                         | 00000011   -       unused                                    | 10000011   PCXYD   request PC XY and direction fields   Memory |
| 0101 = Context Switcher                 | *FS0 packet will have EOT bit set.                                           | 00000100   HLT     cpu fault with halt                       |                                                                |
| 0110 = Program Counter Control & Memory |                                                                              | 00000101   FLT     cpu fault without halt                    |                                                                |
| 0111 = Game Feature Subsystem           |                                                                              | 0001****   ASF,*    auto-select flag (0-15)                  |                                                                |
| 1000 =                                  | FS3 = flags 12-15       X, R**, P**, T**                                     | 00100***   COND,*   conditional opcode (0-7)                 |                                                                |
| 1001 =                                  | FS2 = flags 8-11       M, W, O, S                                            | 00101***   -       unused (0-7)                              |                                                                |
| 1010 =                                  | FS1 = flags 4-7        V, C, A, B                                            |                                                              |                                                                |
| 1011 =                                  | FS0 = flags 0-3        D, F, Z, L                                            |                                                              |                                                                |
| 1100 =                                  | ** Generated or static flags, update is ignored.                             |                                                              |                                                                |
| 1101 =                                  | Byte format (bits 7-0): F3M,F3S,F2M,F2S,F1M,F1S,F0M,F0S                      |                                                              |                                                                |
| 1110 =                                  | F*M are flag mask bits, F*S are flag state bits                              |                                                              |                                                                |
| 1111 =                                  | A set flag mask bit indicates a flag should be updated with the state value. |                                                              |                                                                |

Network functionality:

The omega network allows both persistent "stream" connections and packetized traffic flow modes (but not simultaneously). The streaming mode electrically connects the source and destination without buffers, registers, or flip-flops and can be used for either asynchronous or synchronous communications at arbitrary clock speeds. Depending on the FPGA, it should have <10ns end-to-end signal latency if designed properly. Both stream and packet modes have source-to-destination broadcasts allowing a source address to communicate with all destination addresses. This CPU will likely not use the stream packet type but it will be designed into the omega network module for completeness. There are lots of papers on the Internet discussing the theory behind omega network designs, for more information.

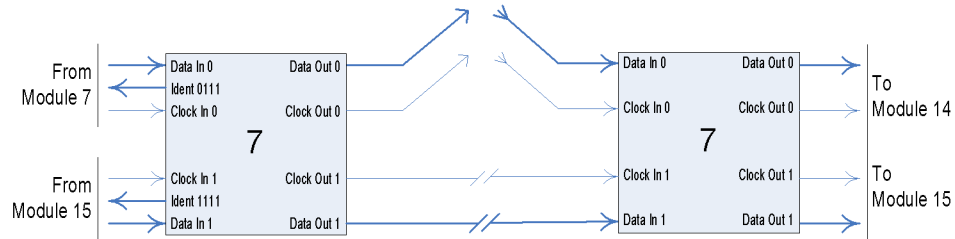
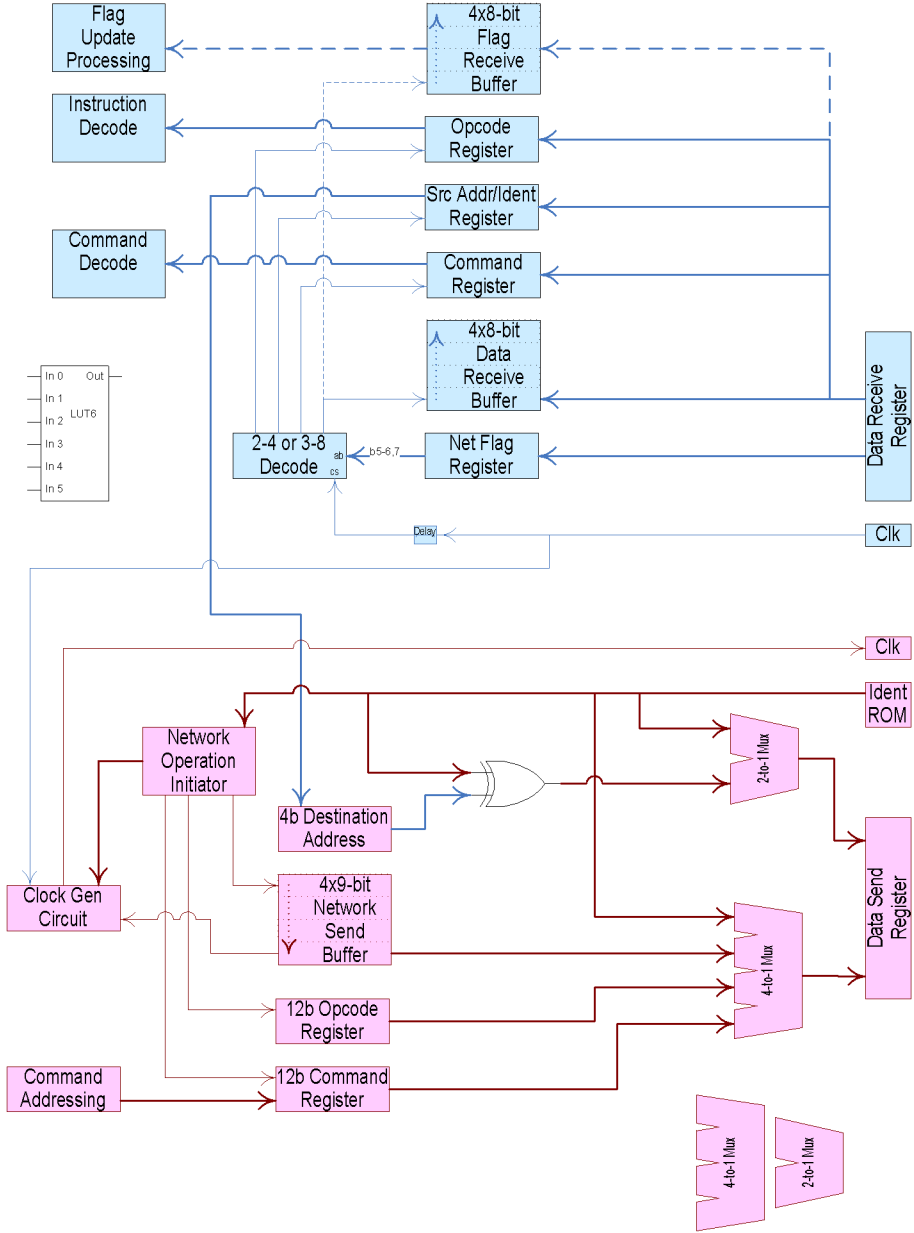
More brief details:

1. Transmission type "stream broadcast":
  - 1a. Broadcast/stream packet type opens a persistent fanout connection on the network. The network links include bi-directional destination-to-source links that are logically OR'ed together and flow back to the broadcast source.
  - 1b. A source address can change the broadcast connection into a point-to-point by sending out a packet of connection type "collapse broadcast", with the destination or XOR tag in the address field. This effectively causes the nodes to recursively disconnect unnecessary links, resulting in a point-to-point stream connection with just the source and proper destination connected. This will rely on some level of application protocol (such as opcode decoding) to determine the proper exclusive destination.
  - 1c. In stream modes, the "disconnect" bit is used by the source to sever the stream link. The EOT bit is not an alternative to the "disconnect" bit and is only for protocol-level functionality.
2. Transmission type "packet":
  - 2a. Broadcasts and "single-destination" connection types are allowed, but "collapse broadcast" and "disconnect" are not used.
  - 2b. An ident packet is a link packet type with an XOR-tag address. This allows a specific destination that "answers" a broadcast to provide its identity to the source address.
  - 2c. An EOT flag is primarily used to latch registers or as an application-level end of communication. It can be used with any of the packet types, although its functionality with the "link" packet type is ignored.
3. Address tag:
  - 3a. Broadcast transmission types (both packet and stream) will always specify the source address in the address field.
  - 3b. "Disconnect stream" packets do not use addressing.
  - 3b. Everything else will always use XOR tag addressing (tag-address = source XOR destination).

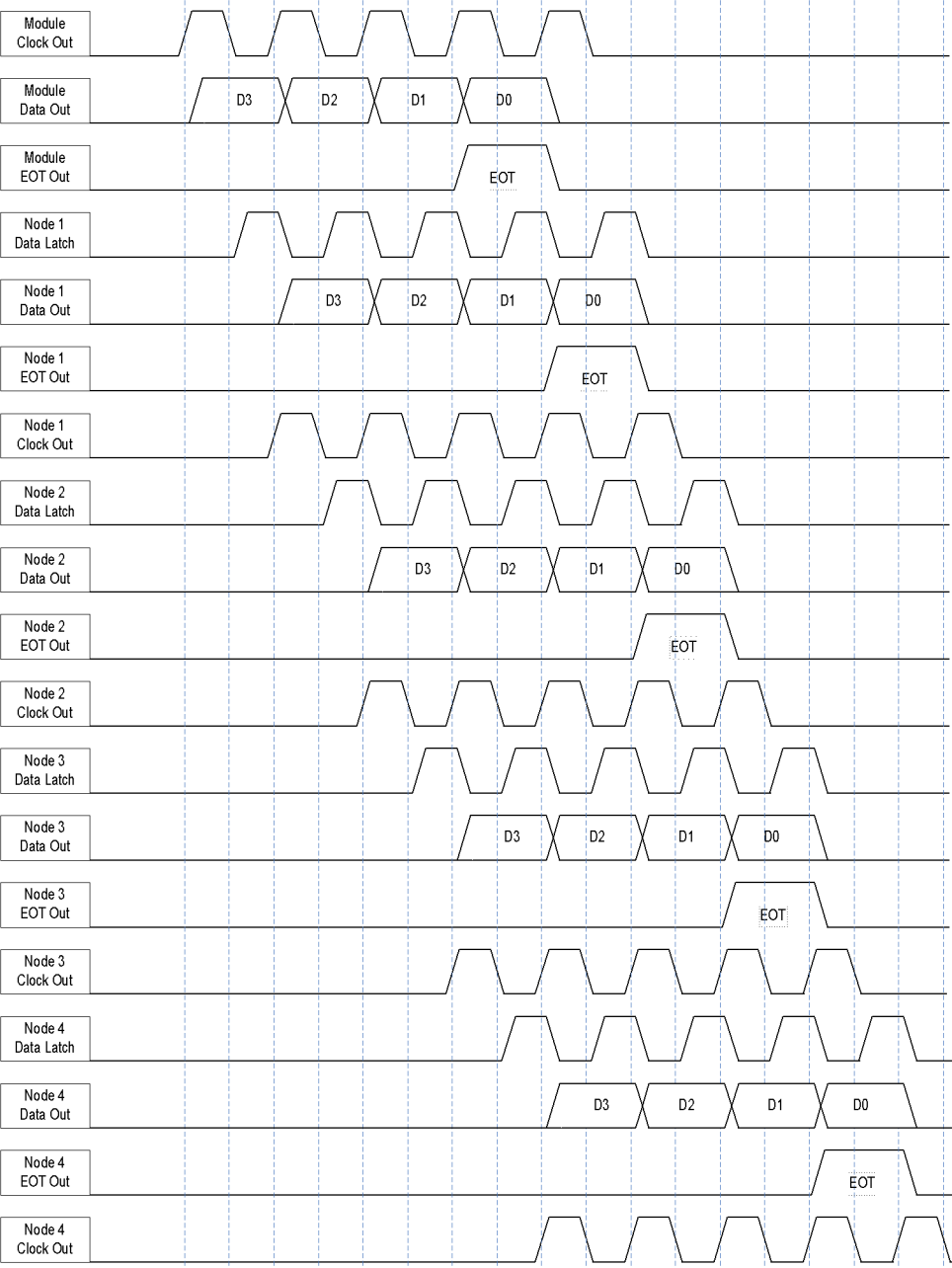
| Omega network source signaling |                                                           |
|--------------------------------|-----------------------------------------------------------|
| b16:                           | Transmission type: 0=single-destination, 1=broadcast      |
| b12-b15:                       | XOR Address (non-broadcast) or Source address (broadcast) |
| b9-b11:                        | Connection type:                                          |
| 000                            | =Link packet                                              |
| 001                            | =Data packet                                              |
| 010                            | =Command packet                                           |
| 011                            | =Flag update                                              |
| 100                            | =Opcode packet                                            |
| 101                            | =Program Counter packet (?)                               |
| 110                            | =Extended Process Info. Packet (?)                        |
| 111                            | =unused                                                   |
| b8:                            | EOT flag                                                  |
| b0-b7:                         | Single-byte payload                                       |

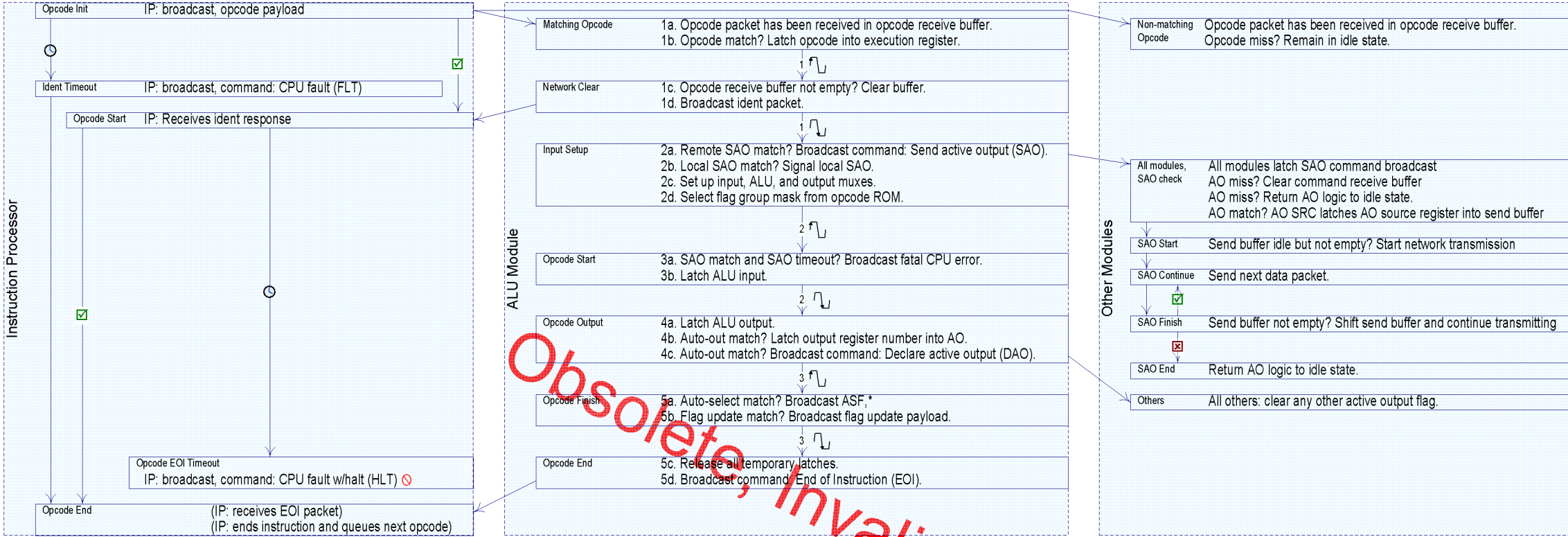
|        |                                                                   |
|--------|-------------------------------------------------------------------|
| b0-b7: | Link payload (for connection type 000):                           |
| b7:    | initiate stream connection 1=yes, 0=no                            |
| b6:    | disconnect stream 1=yes 0=no                                      |
| b5:    | collapse broadcast 1=yes 0=no                                     |
| b4:    | stream clock source: 0=source, 1=destination (non-broadcast only) |
| b0-b3: | Aux. address field.                                               |

|                                 |                     |
|---------------------------------|---------------------|
| Return path (stream mode only): |                     |
| b8:                             | EOT flag            |
| b0-7:                           | Single-byte payload |



- Packet transfers:
1. Data/address word is presented to the data bus of the first network node.
  2. On the first rising clock edge, the data is latched on the first node.
  3. Once the data is latched, the routing logic sets up the multiplexers for further distribution of the packet.
  4. Once the multiplexers are established, the data/address word flows through the enabled multiplexer(s) and onto the second node(s).
  5. On the first falling clock edge, the data is latched on the second node and the first node latch is released.
  6. Once the data is latched, the routing logic sets up the multiplexers for further distribution of the packet.
  7. Once the multiplexers are established, the data/address word flows through the enabled multiplexer(s) and onto the third node(s).
  8. On the rising clock edge, the data is latched on the third node and the second node latch is released.
  9. Steps 3-8 are repeated for the third and fourth nodes.
  10. On the third rising clock edge, the data is latched or shifted onto the module register specified by data bits 6 and 7 and the EOT bit state is latched. The fourth node latch is released.
  11. If more data needs to be transferred, the next trailing clock edge signals that the data should be presented to the network data output and steps 1-10 are repeated. The final clock edge should have the EOT bit set.





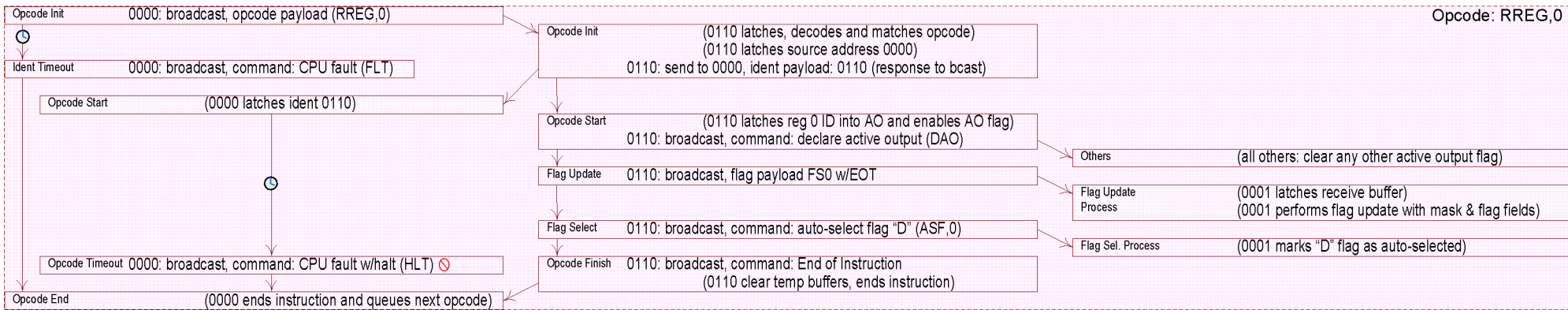
1. Opcode packet and source has been latched.  
1a. Decode opcode and check for match.  
1b. Upon match, transmit ident packet to source.

2. Next clock cycle:  
2a. Latch opcode into execution register.  
2b. Flag if SAO is required.  
2ba: Broadcast command: Send active output (SAO).  
2c. Set up input, ALU, and output muxes.  
2d. Flag if auto-out is required.  
2e. Select flag group mask from opcode rom.  
2f. Flag if auto-select-flag is needed.

3. Next clock cycle:  
3a. Broadcast fault if SAO timeout and halt.  
3b. Latch ALU input.

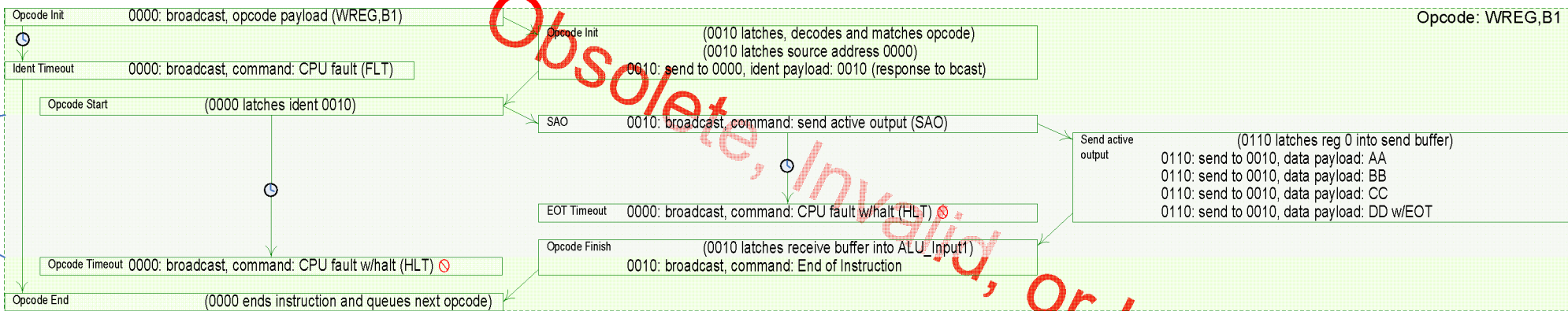
4. Next clock cycle:  
4a. Latch ALU output  
4b. If AO flag is enabled:  
4ba: latch output register number into AO  
4bb: Broadcast command: Declare active output (DAO).  
5. If ASF is valid:  
5a. Broadcast command: ASF,\*  
6. If flag update is required:  
6a. Broadcast flag update byte 1.  
6b. Broadcast flag update byte 2 w/ if EOI.  
7. Broadcast command: End of instruction (EOI)

Example opcode microcode execution flow with direct register transfers. Opcodes: RREG,0; WREG,B1; INC  
(Read register GP0, write to ALU Input B1, perform increment, select ALU output B0 for output)



1. Instruction Processor broadcasts opcode to modules.
- 1a. All modules decode opcode and check for matches.
- 1b. Upon match, register module records source address.
- 1c. Register module notifies broadcaster of opcode accept.

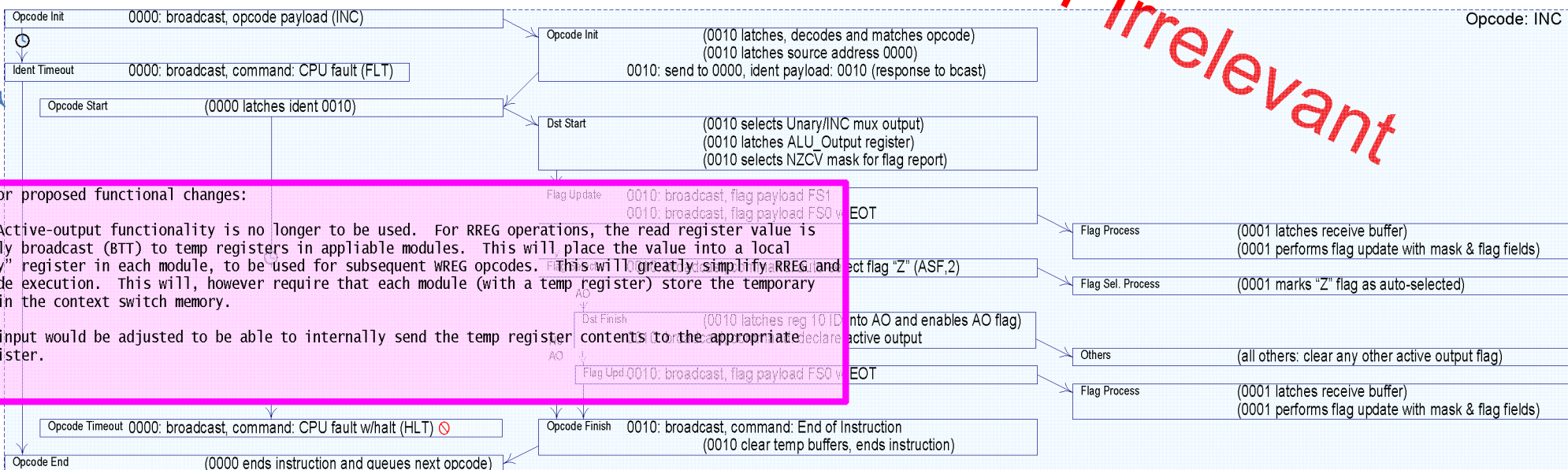
- 1d. Broadcaster records identity of register module.
2. Register module records register 0 as active output.
- 2a. Register module broadcasts active output declaration.
- 2b. All other modules clear any prior active output settings.
3. Register module broadcasts update of "D" flag status.
- 3a. Flag subsystem receives status.
- 3b. Flag subsystem updates "D" flag.
- 3c. Register module broadcasts "D" flag as auto-selected.
- 3d. Flag subsystem marks "D" flag as auto-selected.
4. Register module broadcasts end of instruction.
- 4a. Register module performs clean up.
- 4b. Instruction Processor queues up next opcode.



5. Instruction Processor broadcasts opcode to modules.
- 5a. All modules decode opcode and check for matches.
- 5b. Upon match, ALU module records source address.
- 5c. ALU module notifies broadcaster of opcode accept.

- 5d. Broadcaster records identity of ALU module.
6. ALU module broadcasts send active output request.
- 6a. Register module prepares register 0 data to be sent.
- 6b. Register module transmits uppermost byte.
- 6c. Register module transmits second upper byte.
- 6d. Register module transmits second lower byte.
- 6e. Register module transmits lowermost byte with EOT flag.

7. ALU module transfers received data into ALU Input1
- 7a. ALU module broadcasts end of instruction.
- 7b. Instruction Processor queues up next opcode.



8. Instruction Processor broadcasts opcode to modules.
- 8a. All modules decode opcode and check for matches.
- 8b. Upon match, ALU module records source address.
- 8c. Register module notifies broadcaster of opcode accept.

- 8d. Broadcaster records identity of ALU module.
9. ALU sets up multiplexers to perform INC math operation.
- 9a. ALU transfers INC math result into ALU output register.
- 9b. ALU selects mask of flags affected by math operation.

10. ALU broadcasts flag update of N and Z flags status.
- 10a. ALU broadcasts flag update of C and V flags status.
- 10b. Flag subsystem receives status.
- 10c. Flag subsystem updates requested flags.
- 10d. ALU module broadcasts "Z" flag as auto-selected.
- 10e. Flag subsystem marks "Z" flag as auto-selected.

11. ALU module records register 0 as active output.
- 11a. ALU module broadcasts active output declaration.
- 11b. All other modules clear any prior active output settings.
12. Register module broadcasts update of "D" flag status.
- 12a. Flag subsystem receives status.
- 12b. Flag subsystem updates "D" flag.
13. ALU module broadcasts end of instruction.
- 13a. ALU module performs cleanup.
- 13b. Instruction Processor queues up next opcode.

Note: Major proposed functional changes:

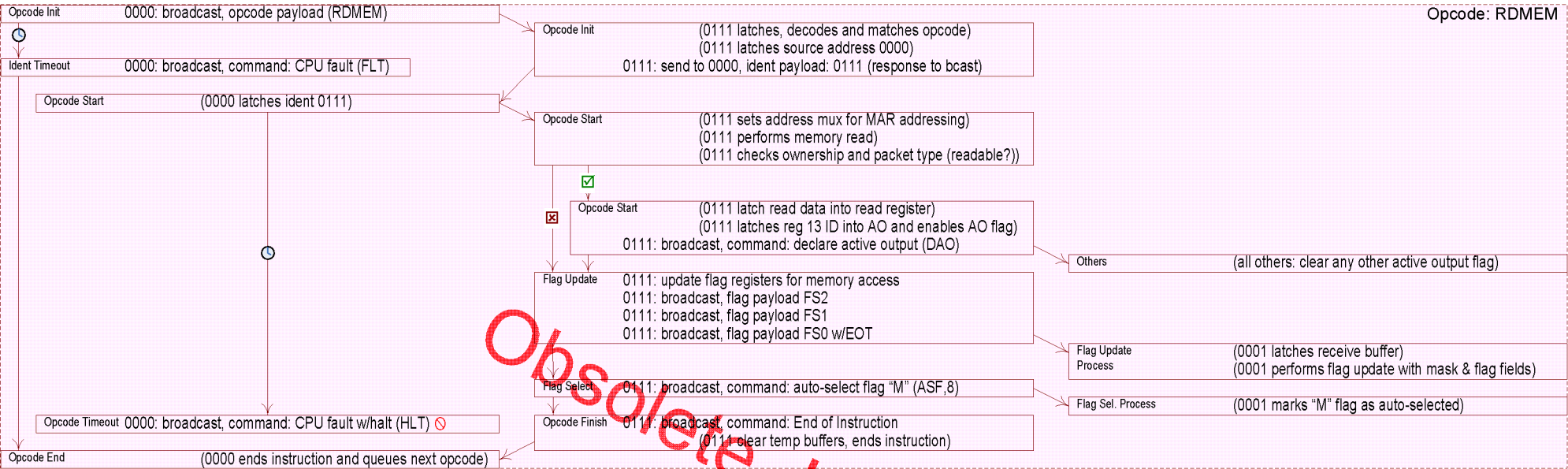
The 'AO' Active-output functionality is no longer to be used. For RREG operations, the read register value is immediately broadcast (BTT) to temp registers in applicable modules. This will place the value into a local "temporary" register in each module, to be used for subsequent WREG opcodes. This will greatly simplify RREG and WREG opcode execution. This will, however require that each module (with a temp register) store the temporary register in the context switch memory.

ALU auto-input would be adjusted to be able to internally send the temp register contents to the appropriate declare active output input register.

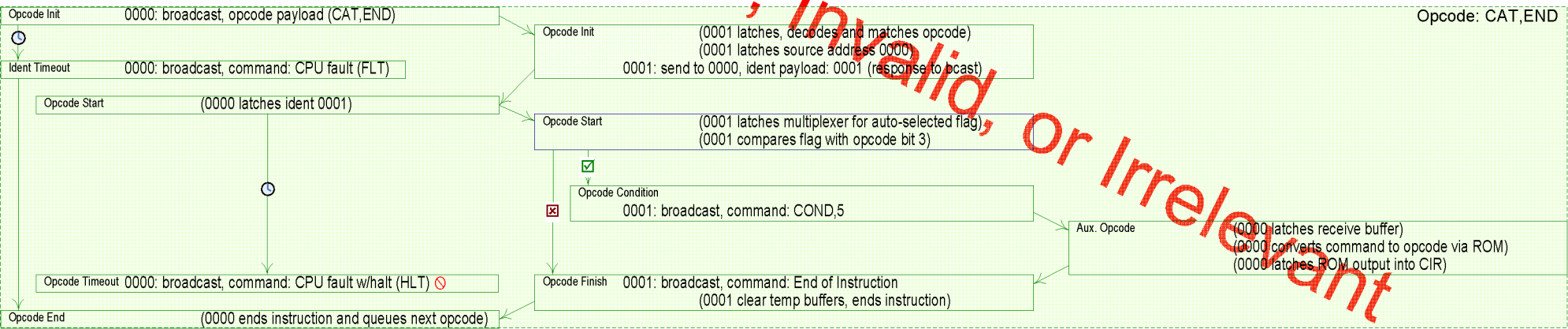
\* If the appropriate "input auto-source" was enabled in the ALU configuration (b12-13=01), then these steps would run as part of the ALU INC opcode, and this WREG,B1 opcode would not be necessary.

\*\* If "auto-output" was disabled in the ALU configuration, then an additional RREG,10 opcode would be needed to perform these steps.

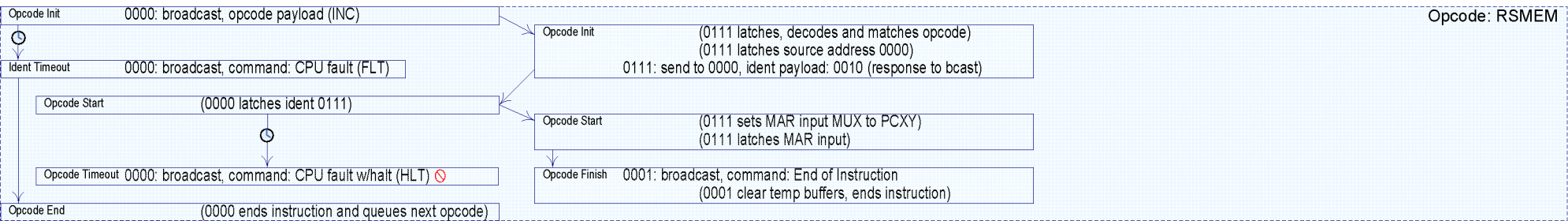
Example opcode microcode execution flow. Opcodes: RDMEM; CAT,END; RSMEM  
(Read memory location; Run conditional END opcode if auto-selected flag is true; reset memory address register to PC location if not)



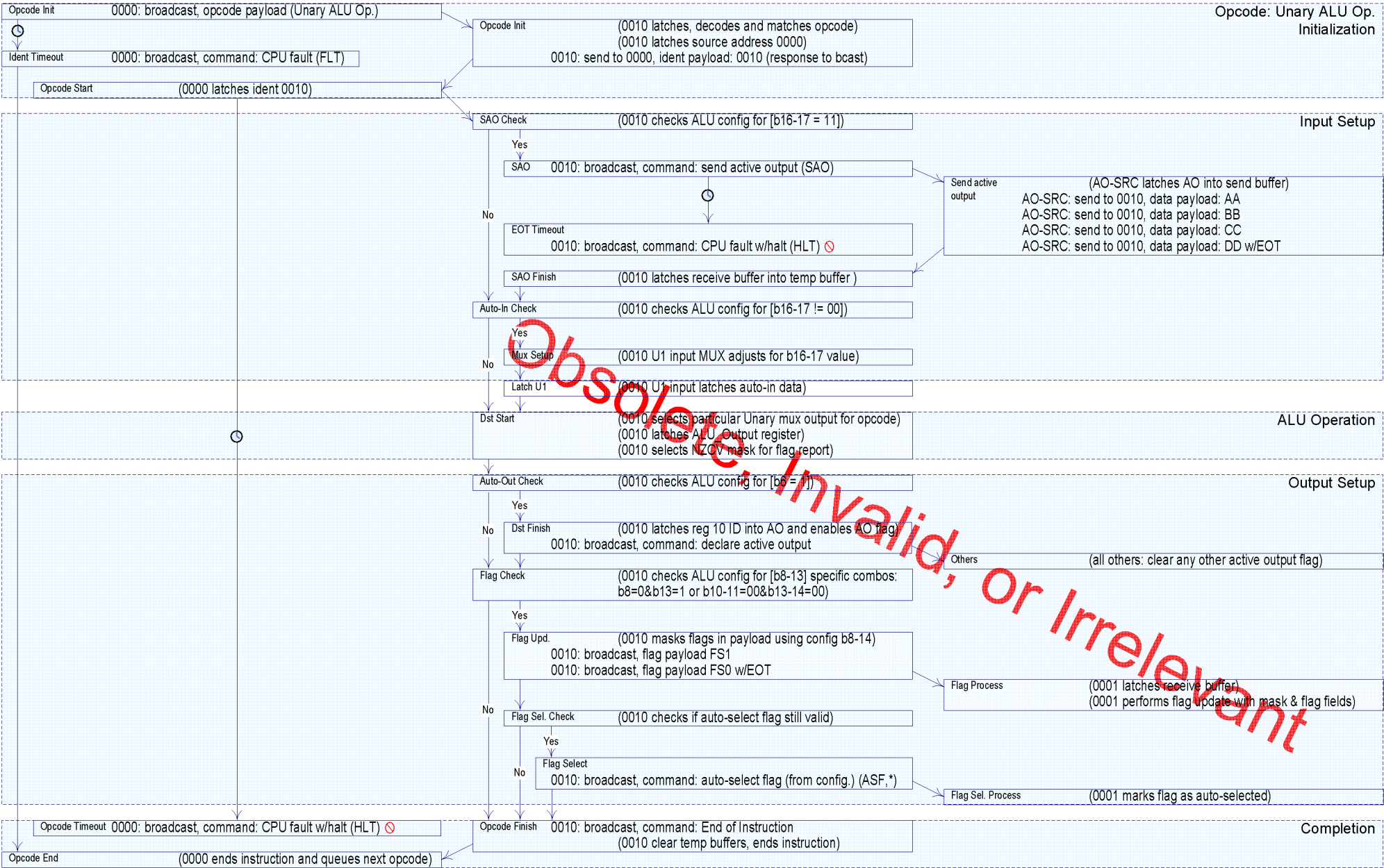
1. Instruction Processor broadcasts opcode to modules.
- 1a. All modules decode opcode and check for matches.
- 1b. Upon match, memory module records source address.
- 1c. Memory module notifies broadcaster of opcode accept.
- 1d. Broadcaster records identity of register module.
2. Memory module sets up multiplexers for MAR addressing
- 2a. Memory location is read.
- 2b. Extended data is checked for allowed access
- 2c. Data value is latched into read register.
3. Memory module records memory register as active output.
- 3a. Memory module broadcasts active output declaration.
- 3b. All other modules clear any prior active output settings.
4. Memory module updates flag registers for new access.
- 4a. Memory module broadcasts update of M/W/O flags status.
- 4b. Memory module broadcasts second flag byte.
- 4c. Memory module broadcasts first flag byte with EOT flag.
- 4d. Flag subsystem receives status.
- 4e. Flag subsystem updates flags M, W, and O.
- 4f. Memory module broadcasts "M" flag as auto-selected.
- 4g. Flag subsystem marks "M" flag as auto-selected.
5. Memory module broadcasts end of instruction.
- 5a. Memory module performs clean up.
- 6b. Instruction Processor queues up next opcode.



7. Instruction Processor broadcasts opcode to modules.
- 7a. All modules decode opcode and check for matches.
- 7b. Upon match, flag module records source address.
- 7c. Register module notifies broadcaster of opcode accept.
- 7d. Broadcaster records identity of flag module.
8. Flag module sets multiplexers for auto-selected flag.
- 8a. Flag module compares flag with T/F bit from opcode.
- 8b. If flag is true, flag module transmits END command.
9. Instruction processor receives command.
- 9a. I.P. converts conditional command to opcode value.
- 9b. I.P. transfers opcode into conditional instruction register.
10. I.P. module broadcasts end of instruction.
- 10a. I.P. module performs clean up.
- 10b. Instruction Processor queues up 'END' or next opcode.



11. Instruction Processor broadcasts opcode to modules.
- 11a. All modules decode opcode and check for matches.
- 11b. Upon match, ALU module records source address.
- 11c. Register module notifies broadcaster of opcode accept.
- 11d. Broadcaster records identity of ALU module.
12. Memory module selects PCXY reg for MAR input MUX
- 12a. Memory module transfers PCXY to MAR
13. ALU module broadcasts end of instruction.
- 13a. ALU module performs cleanup.
- 13b. Instruction Processor queues up next opcode.



1. Instruction Processor broadcasts opcode to modules.  
1a. All modules decode opcode and check for matches.  
1b. Upon match, ALU module records source address.  
1c. Register module notifies broadcaster of opcode accept.

1d. Broadcaster records identity of ALU module.

2. ALU module checks ALU\_U1 AI config for AO setting.

2a. ALU module broadcasts send active output request.  
2b. Register module prepares register 0 data to be sent.  
2c. Register module transmits uppermost byte.  
2d. Register module transmits second upper byte.  
2e. Register module transmits second lower byte.  
2f. Register module transmits lowermost byte with EOT flag.

2g. ALU module transfers received data into temp buffer.

3. ALU module checks ALU\_U1 AI config setting for nonzero.

3a. ALU module sets input MUX for auto-in source.

3b. ALU\_U1 latches auto-in data.

4. ALU sets up multiplexers to perform math operation.  
4a. ALU transfers math result into ALU output register.  
4b. ALU selects mask of flags affected by math operation.

5. ALU module checks ALU\_UO for auto-out enable setting.

5a. ALU module records register 0 as active output.  
5b. ALU module broadcasts active output declaration.  
5c. All other modules clear any prior active output settings.  
6. ALU module checks ALU\_U1 flag config for allowed flags.

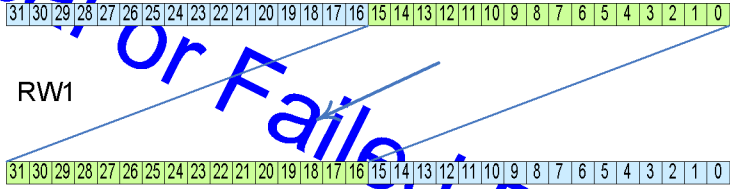
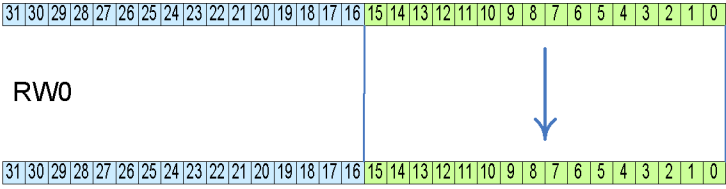
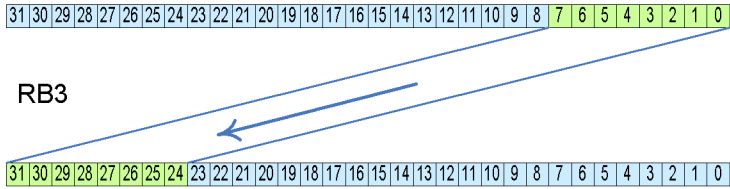
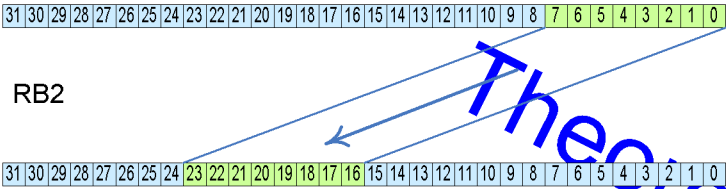
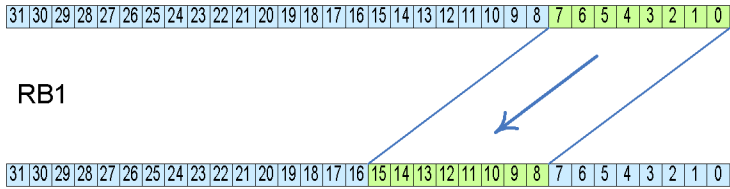
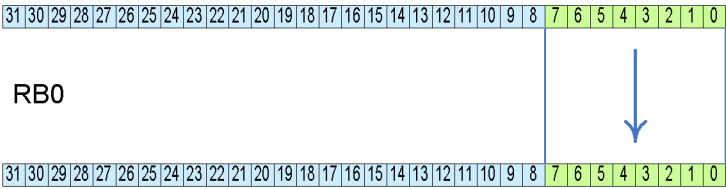
6a. ALU module uses ALU\_U1 flag config to mask flags.  
6b. ALU module broadcasts flag update byte 1.  
6c. ALU module broadcasts flag update byte 2 with EOT.  
6d. Flag subsystem receives status.  
6e. Flag subsystem updates specified flag.  
6f. Check if ALU config still allows auto-select mask.

6g. ALU broadcasts flag as auto-selected based on config.  
6h. Flag subsystem marks flag as auto-selected.

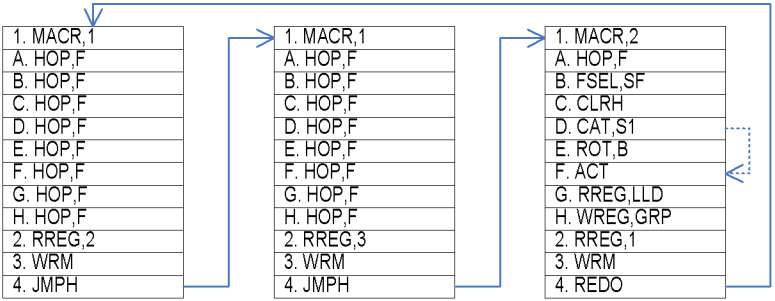
7. ALU module broadcasts end of instruction.  
7a. ALU module performs cleanup.  
7b. Instruction Processor queues up next opcode.

Error

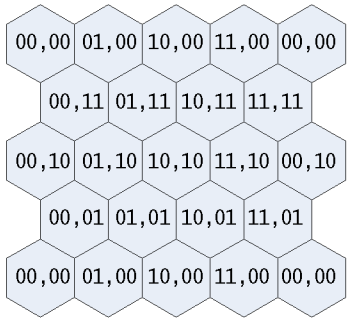




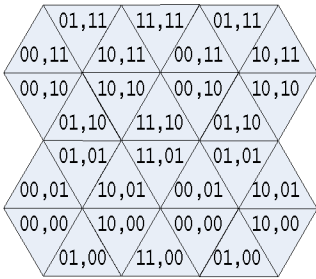
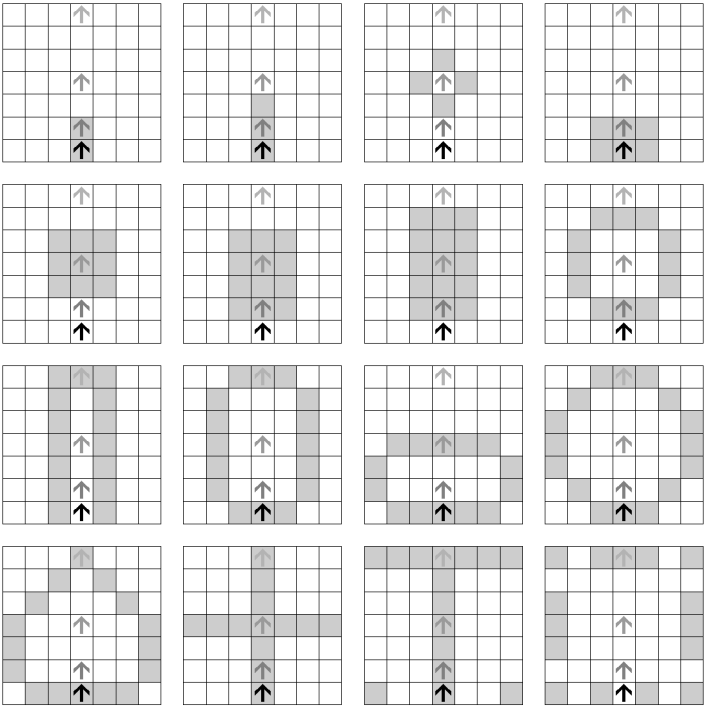
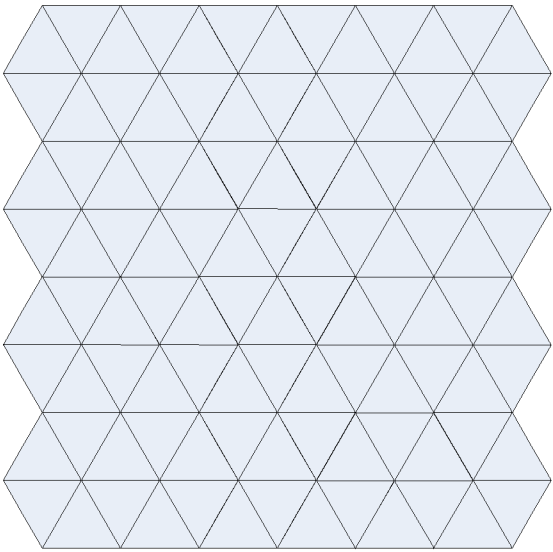
Theoretical or Failed Designs – Not Planned



| Reg 1:    | Reg 2:    | Reg 3:    | Macro 1: | Macro 2:    |
|-----------|-----------|-----------|----------|-------------|
| 1. MACR,1 | 1. MACR,1 | 1. MACR,2 | A. HOP,F | A. HOP,F    |
| 2. RREG,2 | 2. RREG,3 | 2. RREG,1 | B. HOP,F | B. FSEL,SF  |
| 3. WRM    | 3. WRM    | 3. WRM    | C. HOP,F | C. CLRH     |
| 4. JMPH   | 4. JMPH   | 4. JMPH   | D. HOP,F | D. CAT,S1   |
|           |           |           | E. HOP,F | E. ROT,B    |
|           |           |           | F. HOP,F | F. ACT      |
|           |           |           | G. HOP,F | G. RREG,LLD |
|           |           |           | H. HOP,F | H. WREG,GRP |



Coordinate  
System



Coordinate  
System

